Four Artificial Intelligence Paradigms : A Controlled Comparison of Search, Constraint Reasoning, Optimisation, and Reinforcement Learning on the Dhaka Road Network

Papry Rahman
Department of Computer Science and Engineering
University of Dhaka, Dhaka 1000, Bangladesh
papry-2021811177@cs.du.ac.bd

July 25, 2026

Abstract. This study investigates four Artificial Intelligence paradigms on a single real-world substrate: the Dhaka road network, extracted from OpenStreetMap with OSMnx. The domain is held fixed and the paradigm varied, so that each method's strengths and failure modes are exposed on comparable, geographically grounded instances. Route finding under a specific factors, context-aware cost model shows that the efficiency of A^* is governed almost entirely by how faithfully its heuristic mirrors the true multi-criteria cost. Fuel-crisis gas allocation, posed as a constraint satisfaction and optimisation problem, locates the scale at which systematic backtracking must yield to Min-Conflicts local search. Surveillance-camera placement, posed as a directional maximal covering location problem, isolates submodularity as the property that decides whether a metaheuristic beats a greedy baseline. Stochastic multi-stop delivery, posed as a Markov decision process, prices the compute premium a model-free learner pays for lacking a model. Throughout, the appropriate algorithm is dictated by problem structure, and every simulated parameter is a stated design choice rather than a fitted measurement. All source code and the LaTeX report are publicly available at https://github.com/USERNAME/dhaka-ai-paradigms.

Keywords: Artificial Intelligence; heuristic search; constraint satisfaction; swarm optimization; reinforcement learning; Markov decision process; OpenStreetMap

1. Introduction

Artificial Intelligence offers a family of problem-solving paradigms that differ less in the domains they are applied to than in the structure of the problem they presuppose. Systematic search assumes a known, deterministic transition model and seeks a least-cost path. Constraint reasoning assumes a factored state described by variables, domains, and constraints, and seeks a complete consistent assignment. Population-based and swarm optimisation assume a large, non-convex, possibly black-box objective and seek a high-quality configuration. Reinforcement learning assumes a stochastic environment and seeks a policy that maximises expected long-term reward [37]. Choosing among them is therefore not a matter of taste but of diagnosing which structural assumption the problem at hand actually satisfies.

This study makes that diagnosis empirical by holding the domain fixed and varying the paradigm. All four investigations operate on the Dhaka street network extracted with OSMnx [4] ($\approx 1.19 \times 10^5$ intersections and 2.92×10^5 road segments), a substrate chosen because it is real rather than synthetic, and because Dhaka exhibits, in an unusually acute form, exactly the frictions that make each paradigm's assumptions bite: congestion severe enough to collapse average driving speed to a crawl [50], acute fuel rationing under policy constraints [48], a growing municipal surveillance network [42], and monsoon waterlogging that renders streets impassable

within minutes [47].

The four investigations form a natural progression in what is being sought:

1. a best *path* through a known, deterministic cost surface the *routing study* (Section 3);
2. a valid *assignment* of users to resources under hard policy constraints the *rationing study* (Section 4);
3. an optimal *placement* of a fixed sensor budget under a possibly non-submodular objective the *surveillance study* (Section 5);
4. an optimal *policy* under genuine stochasticity the *delivery study* (Section 6).

Each is implemented in Python with an object-oriented design, uses real map data rather than a synthetic toy graph, and is evaluated through controlled parameter sweeps that isolate one factor at a time. Where openly available Dhaka data does not exist at the granularity required edge-level traffic, crime, crash risk, station capacity, hazard placement the corresponding quantity is synthesised from a transparent, qualitatively motivated design choice, stated explicitly and replaceable without touching the solver or evaluation code. Consequently every empirical claim made here depends only on the *relative* ordering between methods, never on the absolute magnitude of a simulated constant.

The remainder is organised as follows. Section 2 reviews the theoretical foundations of each paradigm and identifies, for each, the specific gap in the literature that the corresponding investigation addresses. Sections 3, 4, 5, and 6 present the four investigations, each following an identical structure: motivation and formulation, methodology, results, and key takeaways. Section 7 draws the cross-cutting observations together and outlines future work.

2. Theoretical Foundations and Literature Survey

This section reviews the theoretical foundations and the progression of relevant work for each of the four paradigms investigated. Because this is laboratory work, the objective is not to outperform state-of-the-art systems but to reproduce and compare established techniques under controlled settings, and to identify the critical design choices that govern their success under realistic conditions.

2.1. Heuristic and State-Space Search for Route Planning

State-space search formalizes route finding as the exploration of a graph whose nodes are states and whose edges carry traversal costs. Uninformed strategies, such as breadth-first search (BFS), depth-first search (DFS), iterative-deepening DFS (IDDFS, the optimal admissible tree search of Korf [24]), and uniform-cost search (UCS, the graph form of Dijkstra's algorithm [8]), use no domain knowledge beyond the goal test and are distinguished by their completeness, optimality, and complexity bounds [37]. Informed strategies utilize a heuristic function $h(n)$ estimating the cost-to-go, the systematic study of which was established by Pearl [32]. The A* algorithm expands nodes by $f(n) = g(n) + h(n)$ and is optimal whenever h is *admissible* (never overestimates) and consistent [15]. Greedy best-first search uses $h(n)$ alone to trade optimality for speed, whereas Weighted A* inflates the heuristic ($f(n) = g(n) + \omega h(n),\ \omega > 1$) to trade a bounded loss of optimality for faster search times [33]. A central theme in this literature is that the *informativeness* of h determines how few nodes A* must expand; a weak heuristic degrades A* toward UCS.

Traditional route-planning literature evaluates search efficiency and heuristic admissibility on synthetic grid worlds or abstract uniform graphs, assuming static, single-criterion edge weights [37]. While multi-criteria routing has been studied in theoretical domains [38], little empirical work examines how the informativeness of heuristics (and thus node expansion efficiency) degrades on complex, highly non-linear, multi-criteria cost surfaces such as combining real-time

traffic congestion, gender-safety preferences, and transport-mode constraints in a high-density urban environment like Dhaka [50] a setting where safety-aware route recommendation has begun to be explored for the city itself [17], but not from the standpoint of heuristic informativeness. Our first assignment addresses this gap by comparing uninformed UCS with A* and Weighted A* under various passenger profiles on a real OpenStreetMap Dhaka road network [4], isolating how a weak, misaligned heuristic degrades informed search to uniform-cost search.

2.2. Constraint Satisfaction and Optimization in Resource Allocation

A Constraint Satisfaction Problem (CSP) is defined by variables X, domains D, and constraints C, and a solution is a complete, consistent assignment [7, 25, 37]. CSPs support general-purpose reasoning, where *constraint propagation* prunes large portions of the search space before or during search. Arc consistency, enforced by the AC-3 algorithm [26], removes domain values that cannot participate in any consistent solution. Systematic backtracking is made practical by variable- and value-ordering heuristics: Minimum Remaining Values (MRV), the degree heuristic, and Least Constraining Value (LCV), together with forward checking as lightweight inference, whose efficiency gains over naive backtracking were established by Haralick and Elliott [14]. As an alternative, Min-Conflicts is a local-search repair method that starts from a complete assignment and iteratively reduces violations; it scales to very large instances but sacrifices completeness [27]. When soft preferences are added, the CSP becomes a constraint *optimization* problem with an objective to minimize.

Although constraint satisfaction and optimization solvers are well-studied on synthetic, uniform benchmarks (e.g., n-queens or random graph coloring) [7, 25], their scaling properties and transition thresholds are rarely evaluated under highly skewed, prioritized resource constraints that mirror real-world policy rationing. In particular, while constraint programming handles scheduling tasks [2], the precise scaling threshold where local-search repair (Min-Conflicts) outperforms systematic backtracking under varying constraint tightness remains underexplored on real city topologies. Our second assignment fills this gap by formulating Dhaka's fuel rationing problem as a Constraint Satisfaction and Optimization Problem (CSOP), benchmarking systematic backtracking with heuristics against Min-Conflicts to expose this exact transition under increasing problem sizes.

2.3. Taxonomy of Population-Based and Swarm Optimization

Population-based metaheuristics maintain a population of candidate solutions and evolve them toward high fitness. In the literature, these nature-inspired optimization algorithms are generally grouped into four main taxonomic families based on their source of inspiration:

1. *Evolution-Based Algorithms:* These draw on natural selection, reproduction, and genetic recombination. The Genetic Algorithm (GA) applies selection, crossover, and mutation to a population of chromosomes [13, 16, 21]. Differential Evolution (DE) operates over continuous spaces by adding scaled difference vectors between random population members to a third member [40].
2. *Swarm-Intelligence-Based Algorithms:* These model the collective, decentralized behavior of social organisms and animal groups. Ant Colony Optimization (ACO) constructs path-based solutions using pheromone trails reinforced by quality and diminished by evaporation [9, 10]. Particle Swarm Optimization (PSO) moves candidate particles through a continuous space under attraction to personal best and global best positions [6, 22, 39]. Other swarm methods include the Artificial Bee Colony (ABC) algorithm inspired by honeybee foraging [20], the Firefly Algorithm (FA) based on flash attractiveness [51], Cuckoo Search (CS) using Lévy flights [52], the Grey Wolf Optimizer (GWO) modeling social hunting hierarchy [29], and the Whale Optimization Algorithm (WOA) modeling bubble-net hunting behavior [28].
3. *Physics-Based Algorithms:* These model thermodynamic or gravitational processes. Simulated

Annealing (SA) draws an analogy with metallurgical cooling to escape local optima by accepting worse states with a decreasing probability [23]. The Gravitational Search Algorithm (GSA) models Newton's laws of gravity and motion, where candidate solutions attract each other proportionally to their masses [36].

4. *Human-Behaviour-Based Algorithms:* These model human social, cognitive, or learning activities. Teaching-Learning-Based Optimization (TLBO) models teacher-student interactions in a classroom without requiring algorithm-specific control parameter tuning [35]. Harmony Search (HS) models the improvisation process musicians use to find a pleasing harmony [12].

These methods excel on large, non-linear, or black-box objectives where exact methods are intractable—such as the NP-hard Maximal Covering Location Problem (MCLP) [5] that underlies surveillance-camera placement.

In spatial coverage problems, submodular set functions exhibit a diminishing returns property that guarantees a simple greedy heuristic achieves a $1 - 1/e \approx 63\%$ approximation ratio of the optimal solution [30]. However, when complex, non-submodular elements (such as overlapping field-of-view penalties and continuous facing-angle variables) are introduced, the greedy heuristic's performance guarantees degrade. While population-based metaheuristics are frequently proposed for sensor and CCTV networks [1, 11], the literature lacks controlled studies that isolate the exact point of non-submodularity where population-based search outperforms greedy baselines. Our third assignment addresses this gap by evaluating GA, PSO, and ACO against a greedy baseline on a real Dhaka graph, proving that metaheuristics only justify their compute premium in the non-submodular regime.

2.4. Reinforcement Learning for Stochastic Sequential Routing

Reinforcement learning targets sequential decision making under uncertainty, formalized as a Markov Decision Process (MDP) with states, actions, a stochastic transition function $T(s, a, s')$, and rewards [34, 41]. When the transition model is known, dynamic-programming methods such as Value Iteration solve the Bellman optimality equation [3, 18] by iterated sweeps; when it is unknown, Q-Learning learns action values from experience alone [49].

Reinforcement learning studies comparing model-based (Value Iteration) and model-free (Q-learning) agents are heavily concentrated on synthetic grid worlds or abstract toy environments [41]. There is a lack of studies evaluating the exact cost of model-free exploration on realistic road networks subject to stochastic traffic congestion and waterlogging hazards, despite recent interest in reinforcement learning for stochastic routing [19]. Our fourth assignment addresses this gap by placing both Value Iteration and Q-learning agents on an OpenStreetMap Dhaka network under stochastic hazard transitions. This isolates the trade-off of model availability: we demonstrate that while a model-free learner can recover the model-based optimum policy, it requires a 20× premium in experience and compute.

3. Context-Aware Pathfinding via Uninformed and Informed Search Algorithms on Road Networks

3.1. Motivation and Problem Formulation

Route finding in a city such as Dhaka, Bangladesh is an important and timely shortest-path problem to solve as the cost of traversing a road depends not only on its length but also on the current traffic level, the vehicle type, and personal-safety and accident risks that differ between wide arterials and narrow lanes. Capturing this context turns route finding into a multi-criteria path finding problem in which the best path is genuinely scenario-dependent a car during the evening peak on a route that is safe for a solo female traveller is not the same road as the cheapest one for a bus driver in the early morning. We demonstrate this richness as a testbed for heuristic search, if the heuristic is aligned with the true multi-criteria cost, informed methods

gain a large efficiency advantage. The problem is formulated as follows:

Inputs. A directed road graph $G = (V, E)$ of Dhaka, an origin $s \in V$, a destination $t \in V$, and a *scenario* Σ that captures the full context of the traveller: vehicle type (walk / car / bus), road type(narrow/spacious), time slot, gender, whether travelling alone, urgency (*pace*: relaxed / rush), and a preference profile.

Output. A path $\pi = \langle s = n_0, n_1, \ldots, n_k = t \rangle$ of adjacent nodes and its aggregate cost.

Objective. Minimize the scenario-dependent path cost

$$C(\pi \mid \Sigma) \;=\; \sum_{i=0}^{k-1} \mathrm{cost}(e_i \mid \Sigma), \qquad \mathrm{cost}(e \mid \Sigma) \;=\; \kappa(\Sigma) \sum_{j=1}^{5} w_j(\Sigma)\, u_j(e \mid \Sigma), \tag{1}$$

where each $u_j \in [0, 1)$ is a normalized utility of one of five cost *criteria* distance, vehicle penalty, traffic, personal-safety risk, and accident risk w_j are preference weights that sum to one, and κ is a scenario-dependent scale (Section 3.2.1).

The transition model is deterministic and the graph is static during a query. The cost surface mixes distance with traffic and safety, the geometrically shortest route is generally *not* the cheapest this is what makes heuristic design non-trivial and is the central object of study in this assignment. Table 1 summarizes the notation.

Symbol	Meaning
$G = (V, E)$	Dhaka road graph ($\approx 1.19 \times 10^5$ nodes, 2.92×10^5 edges)
Σ	scenario (vehicle, time slot, gender, alone, pace, profile)
$u_j(e \mid \Sigma)$	normalized utility of criterion j on edge e
$w_j(\Sigma)$	preference weight of criterion j ($\sum_j w_j = 1$)
$\kappa(\Sigma)$	scenario cost scale
$g(n)$	path cost from s to n; $h(n)$ heuristic estimate of cost $n \to t$
$f(n) = g(n) + \omega\, h(n)$	priority key (ω=1 for A*, ω>1 for weighted A*)

Table 1: Assignment 1 notation.

We treat this study as a controlled *empirical comparison* of search strategies over a common, realistic scenario distribution. The comparison shows,

First, whether informed with an admissible, cost-aligned heuristic attains the same optimal path cost as uninformed UCS while expanding significantly fewer nodes.

Second, how sensitive informed search is to heuristic *informativeness* specifically, what happens to the expansion count of informed and the solution cost of greedy best-first when the informative reverse-Dijkstra heuristic is replaced by a weak Euclidean one.

Third, across a distribution of randomized scenarios (vehicle, time slot, gender, pace), how the all strategies rank along the joint dimensions of solution quality, search effort, and wall-clock runtime.

3.2. Methodology

3.2.1 Simulation Environment and Cost Model

The road network is obtained from *OpenStreetMap* (OSM), a free, collaboratively edited geographic database of the world's streets, using the OSMnx library [4]. From OSM we extract Dhaka's drivable network as a directed graph in which *nodes* are road intersections (with latitude/longitude) and *edges* are the road segments between them. Each edge carries its true

length in metres and its OSM `highway` class (e.g. `primary`, `secondary`, `residential`), the latter driving all realism terms of the cost model. The traveller context is a `Scenario` object with the fields listed in Table 1 together with five tunable scale coefficients $(\alpha, \beta, \gamma, \delta_{\text{safety}}, w_{\text{acc}})$.

The Cost Function and Its Parameters Each criterion j is first mapped to $[0, 1)$ by a smooth, saturating *exponential utility*

$$u(x; \tau) \;=\; 1 - \exp(-x/\tau)\,, \tag{2}$$

where the temperature τ_j controls sensitivity (small x grows quickly, large x saturates). We write every criterion *symbolically*; all named parameters are collected used in our simulations, in Table 2:

$$u_1(e) = u\big(\,\text{len}(e);\, \tau_1\big), \qquad \text{distance} \tag{3}$$
$$u_2(e) = u\big(\,\max(0, \rho(e) - 1);\, \tau_2\big), \qquad \text{vehicle penalty} \tag{4}$$
$$u_3(e) = u\big(\,\min(\ell_t^{\max},\; b(\text{slot}) + \eta_{\text{jam}});\, \tau_3\big), \qquad \text{traffic} \tag{5}$$
$$u_4(e) = u\big(r_{\text{sec}}(e)\,\sigma;\, \tau_4\big), \qquad \text{personal safety} \tag{6}$$
$$u_5(e) = u\big(\,\min(\ell_a^{\max},\; r_{\text{acc}}(e)\, m(\text{slot}) + \eta_{\text{acc}});\, \tau_5\big), \qquad \text{accident risk} \tag{7}$$

Here ρ is the vehicle penalty factor; $r_{\text{sec}}, r_{\text{acc}}$ are road-type risk proxies (see Section 3.2.2); $b(\cdot), m(\cdot)$ are the time-of-day traffic base and accident multiplier; σ is user safety sensitivity; and $\eta_{\text{jam}}, \eta_{\text{acc}}$ are random jam/accident noise. The per-edge cost is the profile-weighted sum equation (1), scaled by

$$\kappa(\Sigma) \;=\; \alpha\, k_\alpha + \beta\, k_\beta + \gamma\, k_\gamma + \delta_{\text{safety}} + w_{\text{acc}}\, k_a. \tag{8}$$

Note that κ depends only on the scenario Σ, not on the edge, so it is a single positive multiplier applied identically to every edge it fixes the overall numeric magnitude of the cost but leaves the relative ordering of paths, and hence every algorithm comparison, unchanged. The heuristic's relaxed cost $\underline{c}(\cdot)$ (Section 3.2.3) is built from the same expression and therefore inherits the same κ; because $\kappa > 0$ scales both sides equally, it cancels out of the admissibility relation equation (10).

On the parameter values. The symbols in equations (3) to (8) are *design parameters we chose to simulate* realistic Dhaka conditions; and the framework accepts any calibrated substitute. Table 2 and Table 3 states the real-world basis and data source of each criterion.

Symbol	Meaning
$(\tau_1, \ldots, \tau_5)$	utility temperatures (distance, vehicle, traffic, safety, accident)
$b(\text{slot})$	base traffic level by time slot (peak / moderate / off-peak)
$m(\text{slot})$	accident time-of-day multiplier
ρ	vehicle penalty factor (car / bus, narrow vs. wide roads)
$(\eta_{\text{jam}}, \eta_{\text{acc}})$	random per-edge jam / accident noise
$(\ell_t^{\max}, \ell_a^{\max})$	saturation caps (traffic, accident)
σ	user safety sensitivity (gender / alone / pace, road type)
$(k_\alpha, k_\beta, k_\gamma, k_a)$	scale multipliers in equation (8)
$(\alpha, \beta, \gamma, \delta_{\text{safety}}, w_{\text{acc}})$	default scale coefficients

Table 2: Simulation parameters: symbols and meanings only. Their chosen values, and the rationale for each, are given item-by-item in the design justification (Section 3.2.2).

Criterion	Real-world basis	Representation in the model
Distance	actual travel length	**real** OSM edge length (metres)
Vehicle penalty	large vehicles cannot use narrow lanes; cars crawl on them	OSM `highway` class → feasibility (∞) or penalty ρ
Traffic	rush-hour congestion plus unpredictable local jams	time-slot base b(slot) + random jam noise η_{jam}
Personal safety	narrow, isolated streets feel unsafe (especially women travelling alone)	road proxy r_{sec} × user sensitivity σ
Accident risk	fast arterials incur more/severe crashes, worse at peak hours	road proxy r_{acc} × time multiplier m(slot)

Table 3: Alignment of each cost criterion with real-world Dhaka conditions and its data source.

3.2.2 Design Rationale and Empirical Basis

A route-cost model is only as credible as the reasoning behind its terms. Here we describe our design choices and environment.

(i) Time-of-day. Congestion and crash exposure in Dhaka rise sharply during the morning and evening commutes the World Bank reports that average driving speed has fallen from 21 to under 7 km/h, wasting 3.2 million working hours per day [44, 50] so both the base traffic b(slot) and the accident multiplier m(slot) increase during peak windows. We calibrate b(slot) to 0.9, 0.6, and 0.3 for peak, moderate, and off-peak periods respectively a $\sim$3× peak-to-off-peak ratio that is consistent with the documented threefold speed decline. The accident multiplier $m(\text{slot}) \in \{1.35, 1.15, 1.0\}$ reflects a modest but well-documented rise in crash exposure during rush hours [43]. Utility that depends on the state of the world (here, the time slot) is standard decision-theoretic practice [37].

(ii) Dual, opposing road risk. We keep two risk dimensions separate because they move in *opposite* directions by design. Wide arterials carry a higher accident-risk proxy motorways $r_{\text{acc}} = 0.62$, primaries 0.52, driven by higher speeds and more conflict points (national highways account for the largest single share of Bangladesh's road crashes [43]) yet a lower personal-security proxy ($r_{\text{sec}} = 0.18$ for both) because they are open and populated. Narrow service roads invert this: $r_{\text{acc}} = 0.58$ from parked cars and pedestrian conflicts, but $r_{\text{sec}} = 0.78$ reflecting isolated, poorly lit environments. Secondary, tertiary, and residential roads occupy intermediate positions along both axes. Collapsing these two dimensions into a single number would erase the trade-off that makes safety-aware routing genuinely non-trivial.

(iii) Vehicle feasibility and penalty. A bus physically cannot enter a narrow lane, encoded as $+\infty$; a private car is approximately 2.2× slowed on one (combining crawling speed and reversing difficulty); a bus on a non-primary road incurs a moderate penalty $\rho = 1.4$. These encodings follow directly from the OSM `highway` class [4] and represent hard physical constraints, not approximations.

(iv) Context-dependent safety sensitivity. The sensitivity σ is set to 1.0 for a woman travelling alone, 0.45 for a woman with company, and 0.35 for anyone else alone, with a +0.15 additive on narrow roads and a ×0.55 discount under rush pace. Only the *ordinal structure* is asserted a lone traveller weighs safety most; a hurried one least not the precise magnitudes, which are meant to capture qualitatively distinct traveller profiles.

(v) Preference profiles. Four named profiles encode a traveller's priority ordering over the five criteria (distance, vehicle penalty, traffic, personal safety, accident risk) as integer weights normalised to sum to one. A *balanced* profile treats all criteria comparably $(3, 2, 3, 2, 2)$; *fastest* up-weights traffic avoidance $(2, 1, 4, 1, 1)$; *safest* up-weights both risk terms $(2, 1, 2, 4, 4)$; *distance* strongly prioritises path length $(5, 1, 1, 1, 1)$. Switching the active profile shifts the cost surface without altering the road graph or any algorithm.

On the simulation framing. This environment is explicitly a *simulation*: edge-level ground truth for Dhaka traffic, crime, and crash risk is not openly available at the required granularity, so every numeric parameter is a transparent, qualitatively motivated design choice rather than a fitted measurement. The utility temperatures $(\tau_1, \ldots, \tau_5) = (300, 0.6, 0.8, 0.35, 1.0)$ are calibration constants chosen so that each criterion sits in the responsive part of the exponential utility curve: $\tau_1 = 300\,\text{m}$ matches a typical Dhaka road segment, while τ_2–τ_5 act on already-normalised $[0, 1]$ signals and are therefore sub-unit. Per-edge noise $\eta_{\text{jam}} \sim \mathcal{U}(0, 0.5)$ and $\eta_{\text{acc}} \sim \mathcal{U}(0, 0.2)$ inject per-scenario variability while remaining strictly below the base traffic and accident levels, so they *perturb* the cost surface rather than *dominate* it; saturation caps $(\ell_t^{\max}, \ell_a^{\max}) = (1.5, 1.8)$ bound the combined signals after noise is added. The global scale coefficients $(\alpha, \beta, \gamma, \delta_{\text{safety}}, w_{\text{acc}})$ fix only the overall numeric magnitude of κ (yielding $\kappa \approx 809$ under the default scenario) and leave all relative comparisons between algorithms unchanged. The framework is fully modular: any parameter can be replaced by a calibrated real-data estimate without touching the search or evaluation code, and our analysis depends only on the resulting orderings and ratios, not on absolute values.

3.2.3 From Cost Model to Search: The Algorithmic Approach

We first look at how the cost model and heuristic feed into the search, since that is where the seven algorithms differ.

Every algorithm operates on the same graph $G = (V, E)$ and calls the same $\texttt{cost}(e \mid \Sigma)$ function from equation (1) to price each edge it traverses. The accumulated path cost from the start s to the current node n is denoted $g(n)$. This is the *only* cost information available to uninformed methods.

The heuristic Information

Informed methods additionally receive $h(n)$ heuristic estimate of the minimum remaining cost from n to the goal t (Section 3.2.3). Crucially, $h(n)$ is *pre-computed once* before the search begins and is then looked up in $O(1)$ per node. It enters the algorithm at exactly one place: the **priority key** used to order the frontier. The table below shows how changing this key from ignoring h entirely, to using h alone, to combining $g + \omega h$ produces qualitatively different search behaviours:

Every algorithm expands nodes in some priority order, and for the cost-aware methods that order comes from an evaluation function

$$f(n) \;=\; g(n) \;+\; \omega\, h(n), \tag{9}$$

where $g(n)$ is the cost already paid from the start s to node n and $h(n)$ is the estimated cost-to-go. Each method is a special case of this key: UCS uses g alone ($\omega = 0$ in effect), greedy best-first uses h alone (dropping g), A* sets $\omega = 1$, and Weighted A* takes $\omega > 1$; BFS, DFS, and IDDFS ignore both and order by depth instead.

Method	Key	Uses g?	Uses h?
BFS	depth (hop count)	×	×
DFS	stack order (LIFO)	×	×
IDDFS	bounded depth	×	×
UCS	$g(n)$	✓	×
Greedy	$h(n)$	×	✓
A*	$g(n) + h(n)$	✓	✓
Weighted A*	$g(n) + \omega\, h(n)$	✓	✓

The seven algorithms **Breadth-First Search (BFS)** expands nodes by hop count (FIFO queue), ignoring both g and h. It is complete and finds the fewest-edge path, but on our non-uniform cost surface "fewest edges" is less meaningful. On Dhaka it expands $\sim 3.6 \times 10^4$ nodes and returns a cost-suboptimal route.

Depth-First Search (DFS) also ignores g and h, committing to the first deep branch it finds (LIFO stack). It is memory-efficient ($O(bm)$) but neither optimal nor reliable on cyclic graphs. With no cost guidance, it produces a route 64× the optimum on Dhaka, wandering across the entire city (Figure 2b).

Iterative-Deepening DFS (IDDFS) runs depth-limited DFS at thresholds $0, 1, 2, \ldots$, combining DFS's low memory with BFS's completeness guarantee, at the cost of re-expanding shallow nodes. It still uses no cost information. With our certain edge cap, the goal is deeper than the limit in most Dhaka scenarios, so sometimes it *fails to return a path.*

Uniform-Cost Search (UCS) is the first algorithm to use $g(n)$: it expands the node with the lowest cumulative path cost (min-priority queue), equivalent to Dijkstra on our true cost surface. It is complete and cost-optimal, but blind to the goal's location it radiates in all directions.

Greedy Best-First Search uses $h(n)$ only, ignoring $g(n)$ entirely. It rushes toward the goal greedily, which is fast but reckless it can lock onto a road that *looks* close to the goal even if the cost paid so far is already high.

A* Search combines both signals as $f(n) = g(n) + h(n)$: the g term anchors it to reality (cost paid so far) while h directs the frontier toward the goal. When h is admissible and consistent (guaranteed by Section 3.2.3), it is complete and cost-optimal, expanding no node with $f > C^*$. On Dhaka it achieves the same optimum as UCS while expanding only 7,519 nodes.

Weighted A* inflates the heuristic as $f(n) = g(n) + \omega\, h(n)$, $\omega = 1.7$, making the search greedier at the cost of a bounded optimality loss (within $\omega \cdot C^*$). In practice it expands $\sim 2.7 \times 10^3$ nodes at the lowest runtime the best speed/quality trade-off of all seven methods.

Our edge cost is a *non-geometric* function: two roads of the same physical length can differ vastly in cost due to traffic, vehicle type, and safety. A geometry-only Euclidean h is therefore a weak proxy for the true cost-to-go, while our reverse-Dijkstra h- computed on the same cost surface is tight. This is the central empirical question: *how much of A*'s efficiency advantage survives when h is replaced by a weak baseline?* The answer drives the two-setup experiment of Section 3.3.

Heuristic Construction and the Two Experimental Setups For A* (and greedy best-first) to be well-guided, the heuristic $h(n)$ must be *aligned* with the true cost surface. We design and compare two heuristics that differ in exactly this alignment, and use them as the primary experimental control of this assignment.

Setup 1: Informative, cost-aligned heuristic (reverse Dijkstra). We construct an admissible h by first defining a per-edge *lower-bound cost* $\underline{c}(e)$ that strips every uncertain, user-specific, or non-negative additive term from the true edge cost of equation (1): random jam/accident noise is set to zero, vehicle penalty to one, and safety sensitivity σ to zero. Because every removed term is non-negative, the relaxed cost can only be smaller than the true cost on every edge and for every scenario:

$$\underline{c}(e) \;\le\; \mathrm{cost}(e \mid \Sigma) \qquad \forall\, e \in E,\ \forall\, \Sigma. \tag{10}$$

This single inequality is what separates the heuristic's cost model from the true one: equation (1) is the cost the search actually pays, while $\underline{c}(\cdot)$ is a deliberately optimistic under-estimate used only to guide it. Running *reverse* Dijkstra from the goal t on this lower-bound graph (Algorithm 1) yields $h(n) =$ the minimum lower-bound cost from n to t. Since each edge's lower bound never exceeds its true cost, h never overestimates the true cost-to-go it is **admissible**. And because it is a shortest-path distance, it also satisfies the triangle inequality, making it **consistent**. Together, admissibility and consistency guarantee that A* returns a globally optimal path [15]. Crucially, this h incorporates the same cost structure (distance, traffic, accident) as the true surface it is informed about what makes a route expensive.

Setup 2: Geometric baseline (Euclidean distance). As a controlled baseline, we replace h with straight-line (Euclidean) distance between n and the goal t, scaled by a rough distance/-traffic/accident factor. This heuristic ignores vehicle type, personal safety, and the road-type risk proxies that dominate the true cost on Dhaka streets. It is *not* calibrated to the cost surface, so while it happens to be non-negative it may be a very loose lower bound or even mislead informed methods into expanding regions that are geometrically close but cost-far from the goal.

The key comparison. By changing *only* h between setups holding the cost model, graph, and algorithms fixed we isolate the single factor of **heuristic informativeness**. The prediction from theory is clear: a tighter, -aligned h should let A* expand far fewer nodes. The greedy search is an even sharper witness since it uses h alone, a misaligned h directly inflates its solution cost. Section 3.3 quantifies both effects.

Algorithm 1 Setup 1: admissible heuristic h via reverse Dijkstra over per-edge lower bounds $\underline{c}(\cdot)$.

$h[t] \leftarrow 0$; min-heap $Q \leftarrow \{(0, t)\}$
while $Q \neq \emptyset$ **do**
  $(d, u) \leftarrow \textsc{PopMin}(Q)$
  if $d > h[u]$ **then continue**
  end if
  for all v with an edge (v, u) **do** ▷ predecessors of u in the reversed graph
    $w \leftarrow \underline{c}(v, u)$ ▷ zero noise, unit vehicle penalty, $\sigma = 0$
    if $d + w < h[v]$ **then**
      $h[v] \leftarrow d + w$; $\textsc{Push}(Q, (h[v], v))$
    end if
  end for
end while
return h ▷ $h(n) \le$ true cost-to-go for every $\Sigma \Rightarrow$ admissible and consistent

3.2.4 Evaluation Metrics and Experimental Protocol

Our study demonstates,

- **Success**: whether a path was found (a scenario can be infeasible, e.g. a bus boxed in by narrow roads); the *success rate* averages this over scenarios.
- **Path cost** $C(\pi \mid \Sigma)$: the objective of equation (1); the primary quality metric.
- **Distance (m)**: raw path length, a secondary, scenario-independent quality check.
- **Nodes expanded**: distinct nodes moved to the closed set the truest measure of work done.
- **Runtime (ms)**: wall-clock time, the practical cost.

Each algorithm is evaluated over some randomized scenarios (random vehicle, time slot, gender, alone flag, and pace), and metrics are averaged. The key experimental control is a comparison of two *heuristic setups* (Table 4), designed to isolate the single factor of heuristic informativeness while holding the cost model, queries, and algorithms fixed. The workflow is summarized in Figure 1.

	Setup 1 (informative)	**Setup 2 (Geometric baseline)**
Heuristic $h(n)$	reverse-Dijkstra on edge lower bounds	straight-line (Euclidean) distance
Cost terms seen by h	distance, traffic, accident (true structure)	distance only (geometric)
Admissible?	Yes (never overestimates)	Not aligned with true cost
Purpose	realistic, informed guidance	control: what happens when h is uninformed

Table 4: The two experimental setups. Only the heuristic $h(n)$ changes; everything else is held constant.

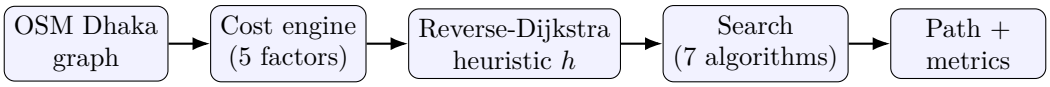


Figure 1: Workflow: a context-aware cost engine feeds an admissible heuristic and a suite of search algorithms.

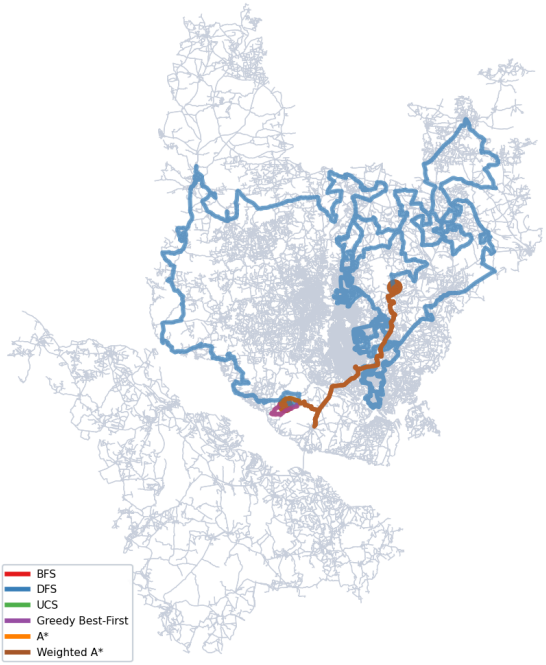
3.3. Results and Discussion

Worked test case

Scenario. Origin: Uttara (23.810, 90.413); destination: Dhanmondi (23.751, 90.393); vehicle: car; time: evening peak (17:00–21:00); profile: balanced.

All cost-aware methods return the *same* optimal route (cost 16,103, length 13.8 km), but expand vastly different numbers of nodes to find it. Greedy and Weighted A* need only 41 nodes; A* needs 196; uninformed UCS and BFS need 6,524 and 7,218 two orders of magnitude more work for the same answer. The same physical path is re-priced at 18,387 under the *fastest* profile and 17,171 under the *safest* profile, showing how the preference weights shift the cost surface without changing the road. Full per-algorithm numbers are in Table 5.

(a) All algorithms on the same scenario: cost-optimal methods (A*, UCS, W-A*, greedy) overlap on one corridor; colours are stacked so only the top-drawn is visible.



(b) DFS (blue) wanders across almost the entire city while all informed methods share a short, direct corridor same scenario, starkly different paths.

Figure 2: Worked test case (Uttara → Dhanmondi, car, evening peak). *Left*: route overlap of cost-aware methods. *Right*: DFS vs. informed methods the cost of ignoring both g and h made visible.

Algorithm	Found	Cost	Expanded	Runtime (ms)
Greedy best-first	yes	16,103	**41**	1.4
Weighted A*	yes	16,103	41	1.3
A*	yes	**16,103**	196	8.0
UCS	yes	**16,103**	6,524	284
BFS	yes	16,103	7,218	130
IDDFS	yes	**16,103**	537,716	12,131
DFS	yes	3,241,926	23,322	563

Table 5: Per-algorithm outcome on the worked test case (single query, Setup 1, balanced profile). The five cost-aware methods return the same optimal route (16,103) with vastly different effort; IDDFS also reaches the optimum here (the goal lies within its 40-edge depth cap) but only after expanding $\sim 5.4 \times 10^5$ nodes over 12 s, and DFS commits to the first deep branch and returns a route $\sim 201\times$ costlier.

Aggregate Analysis:
We now analyse the 10-scenario averages along four dimensions: solution quality, search effort, heuristic informativeness, and runtime together with robustness. Table 6 reports Setup 1.

Solution quality (optimality). The cost column follows theory exactly. UCS and A* are the only optimal methods and return an identical cost of 44,935. Weighted A* (45,474, +1.2%) and greedy (46,328, +3.1%) are mildly sub-optimal, the price of inflating or relying solely on h. BFS (45,943) minimizes the number of hops rather than cost and only accidentally lands near the optimum. DFS is catastrophic at 2.86×10^6 roughly 64× the optimum because it commits to the first path it finds regardless of cost. This cleanly reproduces the completeness/optimality hierarchy of the search literature [37].

Search effort. The headline efficiency result is that A* attains the *same* optimum as UCS while expanding only 7,519 nodes versus 35,496 a 4.7× reduction in work purchased entirely by the heuristic. Greedy and Weighted A* expand the fewest nodes (≈ 2,700) but forfeit optimality; BFS and DFS expand ≈ 36,000, exploring almost the whole reachable graph. Thus expansion count and solution quality trade off along a clear frontier, with A* occupying the optimal-yet-efficient corner.

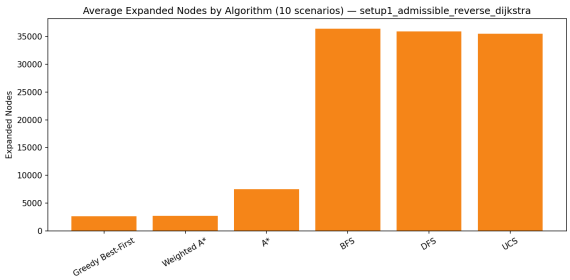
Algorithm	Avg. cost	Expanded	Runtime (ms)	Optimal?
Greedy best-first	46,328	2,679	84	No
Weighted A* (ω=1.7)	45,474	2,696	88	Bounded
A*	**44,935**	7,519	355	**Yes**
UCS	**44,935**	35,496	1,605	**Yes**
BFS	45,943	36,392	708	No (min-hops)
DFS	2,858,809	35,939	751	No
IDDFS	–	–	–	No (depth cap)

Table 6: Assignment 1, Setup 1 (admissible reverse-Dijkstra heuristic). Averages over 10 random scenarios. IDDFS fails on all scenarios because the goal depth exceeds the edge limit.

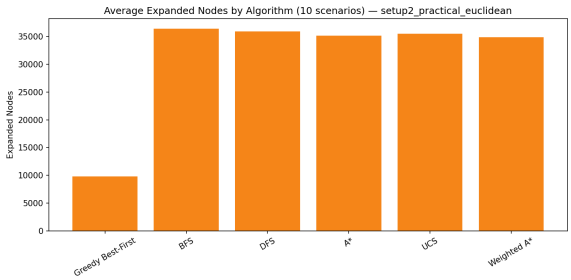
Heuristic informativeness. The role of heuristic quality is isolated by comparing the two setups (Table 7). Under the informative reverse-Dijkstra heuristic, A* expands 7,519 nodes; under the weak Euclidean heuristic blind to traffic and safety, the dominant terms of the cost surface A* expands 35,116 nodes, essentially collapsing onto UCS. The effect on greedy search is even more dramatic: guided by straight-line distance in a jam- and safety-dominated cost space, greedy is pulled along geometrically short but expensive routes and its average cost explodes from 46,328 to 200,686 (4.3× worse). This is the assignment's central empirical claim: *a heuristic misaligned with the true cost forfeits the entire advantage of being informed.* Figure 3 visualizes the expansion collapse.

	Setup 1 (informative h)		**Setup 2** (Euclidean h)	
Algorithm	Avg. cost	Expanded	Avg. cost	Expanded
Greedy	46,328	2,679	200,686	14,830
Weighted A* (ω=1.7)	45,474	2,696	46,201	35,218
A*	**44,935**	7,519	**44,935**	35,116
UCS	**44,935**	35,496	**44,935**	35,496
BFS	45,943	36,392	45,943	36,392
DFS	2,858,809	35,939	2,858,809	35,939
IDDFS	–	–	–	–

Table 7: Setup 1 vs. Setup 2: effect of heuristic informativeness on all seven algorithms. Informed methods (A*, greedy, W-A*) are most affected; uninformed methods are unchanged.



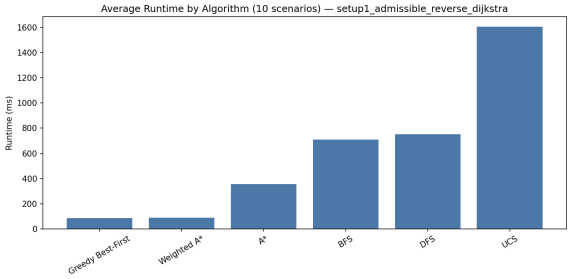
(a) Nodes expanded (Setup 1).



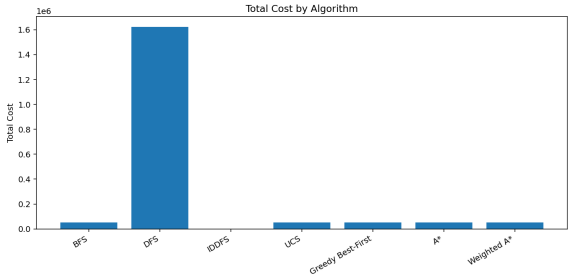
(b) Nodes expanded (Setup 2).

Figure 3: Average nodes expanded per algorithm. A Euclidean heuristic (right) erases the advantage of A* over UCS visible under the informative heuristic (left).

Runtime. Wall-clock time tracks expansion but amplifies it through per-node heuristic look-ups. In Setup 1, A* (355 ms) is 4.5× faster than UCS (1,605 ms); greedy and Weighted A* are fastest (≈ 85 ms). Note that a heuristic is not free: in Setup 2 the added heuristic evaluations make A* (and even Weighted A*) *slower* than in Setup 1 while expanding as many nodes as UCS an informative heuristic pays for itself, a weak one is pure overhead. Figure 4 shows the runtime ranking alongside the solution-cost ranking, in which DFS's cost dwarfs every other method.



(a) Average runtime per algorithm (Setup 1).



(b) Path cost per algorithm (single query).

Figure 4: Runtime (left) and solution cost (right) by algorithm. A* reaches the optimum at a fraction of UCS's runtime, while DFS's path cost is orders of magnitude larger than any other method a visual confirmation of its non-optimality.

Robustness, feasibility, and synthesis. The 70% success rate is not solver failure but genuine scenario infeasibility (e.g. a bus whose origin is surrounded by narrow roads it cannot legally enter); all cost-based solvers agree on feasibility because they share the same $+\infty$-pruned successor function. A different, *structural* failure affects IDDFS: in every sampled scenario the shortest-hop depth to the goal exceeds its 40-edge cap, so it returns no path at all a reminder that a fixed depth limit does not transfer to city-scale graphs. Because all figures are averaged over randomized scenarios, the rankings are stable rather than artefacts of a single query, and across all four dimensions the same lesson recurs: A* with an admissible, *informative* heuristic is the only method that is simultaneously optimal and efficient, and its advantage is exactly proportional to how well the heuristic mirrors the true multi-criteria cost.

3.4. Key Takeaways and Empirical Synthesis

Our empirical evaluation of the multi-criteria Dhaka routing problem yields three definitive conclusions:

1. **Optimal vs. Uninformed Search**: A cost-aware multi-criteria surface renders hop-based

metrics obsolete. BFS and DFS ignore edge weights, leading DFS to catastrophic sub-optimality (64× optimal cost). Only UCS and A* guarantee exact cost optimality, but A* achieves it with a 4.7× reduction in node expansions (7,519 vs. 35,496).
2. **Importance of Heuristic Alignment**: A heuristic's value lies entirely in its fidelity to the underlying cost surface. Replacing our reverse-Dijkstra lower-bound h with a naive Euclidean baseline (Setup 2) causes A* to collapse to blind UCS (35,116 expansions) and degrades Greedy search cost by 4.3× (200,686). Informed search is only as good as its heuristic's alignment with reality.
3. **Practitioner Recommendation**: Weighted A* ($\omega = 1.7$) emerges as the superior operational algorithm for real-time Dhaka navigation. It expands only ≈2,700 nodes (matching Greedy speed at 88 ms) while guaranteeing solution quality within 1.2% of the true optimum (45,474).

While single-agent search solves spatial pathfinding on the urban graph, urban management during crises (such as fuel shortages) introduces multi-agent resource competition with capacity and policy limits. In Section 4, we transition from single-agent spatial search to multi-user global constraint optimization.

4. Urban Resource Allocation and Fuel Rationing under Spatial Policy Constraints

4.1. Motivation and Problem Formulation

Urban fuel crises turn a routine service interaction into a constrained allocation problem where each vehicle has a limited range, rationing policies bar it from many stations, and the city's finite station capacities must be shared equitably among all users simultaneously. Solving this is a timely and realistic problem on urban area because the legality of any one assignment depends on every other assignment station loads and household choices interact globally. We formalize the problem as a Constraint Satisfaction and Optimization Problem (CSOP) [7, 37]:

Variables. One variable per user, $X = \{x_1, \ldots, x_n\}$; assigning a value to x_i decides where user i fuels. Each user carries fixed attributes a location on the Dhaka road graph, a vehicle class, a license-plate parity, a remaining fuel range, and a fuel demand that shape which values are legal for it.

Domains. Every variable draws its value from the set of fueling stations $S = \{s_1, \ldots, s_m\}$, each with a finite capacity; a complete assignment is a map $\sigma : X \to S$, where $\sigma(x_i) = s_j$ means user i fuels at station j. The *effective* domain $D_i \subseteq S$ of each variable is what survives the unary constraints below.

Constraints. A set C of nine hard constraints derived from the government rationing policy Π, grouped by arity: *unary* constraints filter which stations are physically and legally reachable per user; a *binary* anti-hoarding constraint prevents household members from sharing a station; and *global* constraints enforce capacity limits, public-transport reserves, emergency reserves, and fair-service quotas. A full listing with their policy basis appears in Table 9. An assignment is *feasible* iff every constraint in C is satisfied.

Objective. Among feasible assignments making this a constraint satisfaction *and* optimization problem minimize the dual-criteria soft cost

$$\text{cost}(\sigma) \;=\; w_d \sum_{x \in X} \text{dist}\big(x, \sigma(x)\big) \;+\; w_L \,\text{Var}\Big(\{\ell_s(\sigma)\}_{s \in S}\Big), \qquad \ell_s(\sigma) = \frac{\sum_{x \in \sigma^{-1}(s)} \text{demand}(x)}{\text{capacity}(s)}, \tag{11}$$

where the first term penalizes aggregate travel distance and the second penalizes unequal station loading a proxy for preventing localized queue overflows onto surrounding roads. The weights (w_d, w_L) and the distance oracle $\text{dist}(\cdot)$ are scenario-dependent (Section 4.2.1); Table 8 collects

all symbols.

Symbol	Meaning
X, n	set of users / number of users
S, m	set of fueling stations / number of stations
$D_i \subseteq S$	legal-station domain for user x_i after unary filtering
$\sigma(x)$	station assigned to user x
$\ell_s(\sigma)$	demand-to-capacity load ratio at station s
(w_d, w_L)	cost weights for travel distance and load variance
Π	government rationing policy (parity, reserves, caps, quotas)

Table 8: Assignment 2 notation.

We treat this study as a controlled empirical evaluation of CSP-solving strategies, analysed in Section 4.3.

First, whether variable-ordering (MRV, degree), value-ordering (LCV), forward checking, and AC-3 arc consistency convert intractable naive backtracking into a practical solver. **Second**, at what problem scale n systematic backtracking becomes infeasible, and whether local search (Min-Conflicts) extends tractability beyond that point. **Third**, how severely aggressive rationing shrinks the feasible domain, and what fraction of users it renders unserviceable on a city-scale instance.

4.2. Methodology

4.2.1 Simulation Environment and Rationing Model

The world is built from two real data sources. The Dhaka road network is loaded from *OpenStreetMap* via the OSMnx library [4], represented as a directed graph whose *nodes* are intersections and whose *edges* are road segments carrying their true length and OSM `highway` class. Real fueling stations are queried directly from OSM using the tag `amenity=fuel`; because OSM does not record pump capacities, these are synthesized from a parametric distribution (values and rationale in Section 4.2.2).

Each *user* $x_i \in X$ is a synthetic vehicle generated at a random road-network node, characterized by four attributes: a vehicle class (private car, public bus/CNG, or emergency vehicle), a license plate parity (odd or even), a remaining fuel range (determining how far the user can travel to reach a station), and a volumetric fuel demand. Vehicle class splits and demand ranges are design choices motivated by Dhaka's fleet composition; they are stated and justified in Section 4.2.2.

A per-(vehicle-class, station) *distance oracle* is pre-computed by reverse Dijkstra on a vehicle-aware edge weight: road-type restrictions (wide vs. narrow) and a time-of-day traffic scaling are baked into the edge weight so that the oracle reflects true reachability, not straight-line distance. Reachability constraint U1 holds for user x_i at station s iff this oracle distance does not exceed x_i's remaining fuel range. Table 9 lists all nine constraints that filter and shape the feasible space.

ID	Type	Enforces	Policy basis
U1	unary	user can reach station within remaining fuel range	physical reachability
U2	unary	a fraction of stations serve emergency vehicles only	emergency access
G1	unary	odd/even plate day bars off-parity civilian vehicles	2022 rationing [48]
G6	unary	CNG stations close to civilian traffic during peak hours	Dhaka CNG policy
B1	binary	household members must refuel at *different* stations	anti-hoarding
G3	global	reserve a fraction of civilian capacity for public transport	keep buses running
G4	global	lock a fraction of capacity for emergency vehicles	strategic reserve
G8	global	at most $q_{\max}$ users queued per station	avoid road-blocking queues
G9	global	each station must serve at least one user	equitable coverage

Table 9: The nine constraints, grouped by arity, with the real-world rationing lever each encodes. Values and justification appear in Section 4.2.2.

4.2.2 Design Rationale and Empirical Basis

A constraint model is only as credible as the reasoning behind its rules, so we justify each design choice rather than merely assert it.

(i) Plate-parity rationing (G1). Odd/even plate-day restrictions are a well-documented tool for managing acute fuel shortages. During the 2022 Bangladesh fuel crisis, surging global oil prices forced the government to close filling stations one day per week and later impose plate-parity scheduling to reduce queue lengths, which had grown long enough to block surrounding traffic [45, 48]. We encode this as a hard unary filter: on a designated parity day, all off-parity civilian vehicles have their domain pruned to the empty set, forcing the solver to declare them unserviceable an honest representation of the policy's effect.

(ii) Emergency-station exclusivity (U2). We designate a fraction of stations as exclusive to emergency fleets (ambulances, fire engines, police). The motivation is the documented concern that during the 2022 crisis, petrol stations serving mixed traffic created dangerous delays for emergency vehicles [45]. Setting 8% of stations as emergency-only is a conservative estimate; the parameter is fully tunable. This is modelled as a unary filter that removes non-emergency users from those stations' domains before search begins.

(iii) Peak-hour CNG closures (G6). Dhaka's CNG refueling network operates under periodic civilian restrictions during peak commute hours, a policy used to prioritise fuel for the auto-rickshaw and taxi fleet that serves bulk passenger transport [42]. We implement this as a unary constraint that closes CNG-type stations to private vehicles during designated time slots. Because CNG stations constitute approximately 35% of queried OSM stations, this constraint materially shrinks domain sizes.

(iv) Anti-hoarding binary constraint (B1). During fuel crises, household stockpiling is a primary driver of queue overflows. To model the anti-hoarding regulation, vehicles sharing a household cannot be assigned to the same station ($x_i \neq x_j$ for household peers). This is the only binary constraint in the problem and is the reason AC-3 arc consistency is applied: household relationships form a sparse constraint graph that propagates domain pruning across household pairs before search begins.

(v) Capacity reserves: public transport (G3) and emergency (G4). Two global constraints carve out fixed fractions of each station's civilian capacity. Reserving 60% for public-transport vehicles (G3) reflects the government priority of keeping buses and shared taxis

running during a supply shock, since these carry far more passengers per litre than private cars. The 10% strategic emergency reserve (G4) is a standard civil-defence buffer. Both fractions are design choices calibrated to make the constraints clearly binding (not trivially satisfied) on our synthetic fleet; they can be replaced by any empirically derived reserve requirement without changing the solver logic.

(vi) Queue cap (G8) and fair-service quota (G9). Queue lengths at fuel stations during the 2022 crisis were reported to block surrounding roads, disrupting traffic for hours [45]. We enforce a maximum queue of $q_{\max} = 8$ concurrent users per station as a global capacity constraint, chosen to be plausibly tight but not trivially infeasible on our instance sizes. The fair-service quota (G9) requires every station to serve at least one user, preventing degenerate solutions that overload a single central station.

On the simulation framing. Station capacities are synthesized from a $2000\,\mathrm{L} \pm 40\%$ distribution, reflecting realistic variation between small neighbourhood pumps and large arterial stations; the $\pm 40\%$ jitter injects per-scenario variability analogous to the random noise in Assignment 1's cost surface. User vehicle-class proportions (60% private, 30% public, 10% emergency), fuel ranges, and demand levels are calibrated to produce a fleet that stresses all nine constraints simultaneously not to replicate a specific census. The cost weights $(w_d, w_L) = (1, 500)$ are set so that load variance dominates distance: a solution that evenly distributes demand across all stations is always preferred over one that concentrates load even if individual trips are slightly shorter.

Value above station capacities, vehicle-class proportions, fuel ranges, demand levels, the queue cap, and the cost weights is a transparent, tunable design choice, not a measured Dhaka quantity, and each can be replaced by a calibrated real-data estimate without touching the solvers or the evaluation code. As in the earlier assignments, our conclusions therefore depend only on the relative orderings between solver configurations, not on the absolute cost magnitudes.

4.2.3 From Rationing Policy to Fuel Allocation: The Algorithmic Approach

The three solvers share the same preprocessed problem; they differ only in how they search it. We make that split precise before describing each one.

Before any solver runs, two preprocessing passes reduce the domains. First, *unary filtering* removes stations that violate constraints U1, U2, G1, and G6 for each user independently. Second, *AC-3 arc consistency* [26] is applied to the one binary constraint B1 (household anti-hoarding): for every pair of household members (x_i, x_j), any station value $v \in D_i$ that has no compatible counterpart in D_j is pruned. This propagates through household chains and can trigger domain wipe-outs that prove inconsistency before search even starts (Algorithm 2). All three solvers then receive the same *post-filtering* domains D'; the difference lies entirely in *how* they search within them.

Algorithm 2 AC-3 arc consistency for the household not-equal constraint (B1).

```
Q ← all arcs (x_i, x_j) such that x_i, x_j share a household
while Q ≠ ∅ do
    (x_i, x_j) ← DEQUEUE(Q)
    if REVISE(x_i, x_j) then
        if D_i = ∅ then return INCONSISTENT
        end if
        for all x_k in x_i's household, x_k ≠ x_j do
            ENQUEUE(Q, (x_k, x_i))
        end for
    end if
end while
return CONSISTENT

function REVISE(x_i, x_j)
    drop from D_i every v with no w ∈ D_j s.t. w ≠ v;  return whether D_i shrank
end function
```

What the cost function adds and where it enters. The soft objective cost(σ) from equation (11) is *invisible* to systematic backtracking: backtracking returns the *first* complete consistent assignment it finds, not the minimum-cost one. Min-Conflicts, by contrast, directly minimizes a conflict count that acts as a proxy for constraint violations and since the repair move selects the station that *minimizes violations*, it implicitly tends toward load-balanced assignments that also reduce cost(σ). This asymmetry is one reason Min-Conflicts yields slightly lower cost than backtracking on feasible instances (Section 4.3).

The solver configurations. The table below shows what each configuration uses, mirroring the search-key table of Assignment 1:

Solver	Strategy	AC-3	Heuristics	Cost-optimal?
Naive BT	DFS, no ordering	✓		No (first feasible)
Heuristic BT	DFS + MRV/deg/LCV + FC	✓	MRV, degree, LCV	No (first feasible)
Min-Conflicts	local repair + restart	✓*		No (violation proxy)

*AC-3 as pre-filter only.

Variable and value-ordering. In CSP backtracking, the analogous role is played by variable- and value-ordering heuristics: **MRV** (Minimum Remaining Values) selects the variable most constrained by current assignments the one most likely to fail first so dead ends are detected early rather than after many nodes have been expanded. The **degree** heuristic breaks MRV ties by choosing the variable involved in the most constraints on remaining unassigned peers, propagating pruning effects most widely. **LCV** (Least Constraining Value) then orders station choices so that the assigned value rules out the fewest options for other variables the mirror of h in that it looks *forward* to preserve flexibility. Together, these heuristics compress the effective branching factor without changing the correctness guarantee of backtracking.

Now lets focus on the three solver configurations in detail.

Solver 1: Backtracking (naive and heuristic). Systematic depth-first assignment (Algorithm 3). In its *naive* form it assigns values with no ordering: complete if a solution exists it is found but its effort grows as $O(d^n)$ in the worst case, so it is tractable only on small instances and serves as our baseline. The *heuristic* form augments the same DFS with the MRV, degree,

and LCV orderings described just above, plus *forward checking*, which prunes the chosen station from household neighbours' domains and fails early if any domain empties. These heuristics leave completeness and optimality untouched; they only compress the effective branching factor, and how much they compress it on our nine-constraint problem is the first thing the experiments measure.

Algorithm 3 Backtracking search with MRV + degree + LCV + forward checking.

```
function BACKTRACK(σ)
    if σ is complete then return σ
    end if
    x ← SELECTVAR                                   ▷ MRV, tie-broken by degree
    for all s ∈ ORDERVALUES(x) do                   ▷ LCV order
        if s is consistent with σ (B1, G3, G4, G8) then
            add x ↦ s;  FORWARDCHECK (prune neighbours)
            if no domain became empty then
                r ← BACKTRACK(σ);  if r ≠ failure return r
            end if
            undo x ↦ s and its pruning              ▷ backtrack
        end if
    end for
    return failure
end function
```

Solver 2: Min-Conflicts local search. Min-Conflicts (Algorithm 4) abandons systematic search for local repair [27]: from a random complete assignment it repeatedly moves a randomly chosen conflicted user to the station that minimizes the total violation count, restarting on plateaus. It is neither complete nor optimal, but it scales to instances far beyond systematic search. Because its repair move is violation-driven, it implicitly also reduces the load-variance component of cost(σ), so it tends to return slightly lower soft cost than backtracking on feasible instances.

Algorithm 4 Min-Conflicts local search (with random restart).

```
σ ← random complete assignment (each user a random legal station)
for step = 1 to max_steps do
    if VIOLATIONS(σ) = 0 then return σ
    end if
    x ← a randomly chosen user currently in conflict
    σ(x) ← arg min_{s∈D_x} VIOLATIONS(σ with x ↦ s)        ▷ ties random
    on a plateau, perform a random restart
end for
return best-so-far σ
```

Table 10 contrasts the formal properties of the three solver configurations, in the spirit of the complexity table of Assignment 1.

Solver	Strategy	Complete?	Cost-optimal?	Effort / regime
Naive backtracking	systematic DFS, no heuristics	Yes	No (first feasible)	$O(d^n)$ worst case; small n only
Heuristic backtracking	DFS + MRV/degree/LCV + FC + AC-3	Yes	No	far smaller effective branching; medium n
Min-Conflicts	local repair from a random start	No	No	$\sim O(\text{steps}\cdot n\cdot d)$; large n

Table 10: Properties of the three solver configurations (n: users, d: maximum domain size).

4.2.4 Evaluation Metrics and Experimental Protocol

This study records metrics that separate correctness, quality, and effort: **feasibility** (solution found vs. timeout); **cost** equation (11); **search effort**, measured as *backtracks* for systematic search and *repair iterations* for Min-Conflicts; **runtime** (ms); and **users served / unassignable**.

We run three studies plus an ablation: (A) a head-to-head on a small instance; (B) a scaling sweep $n \in \{10, 20, 40, 80, 160\}$; (C) a realistic full-Dhaka instance (150 users, 30 stations); and a *three-way* comparison naive backtracking (heuristics off), heuristic backtracking, and Min-Conflicts on identical instances of increasing size. The overall workflow is summarized in Figure 5.

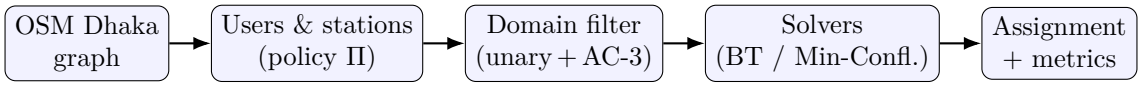


Figure 5: Workflow: users and stations under rationing policy Π are domain-filtered by unary constraints and AC-3, then solved by systematic backtracking or Min-Conflicts local search.

4.3. Results and Discussion

Worked test case

Scenario. A small Dhanmondi scenario consisting of 12 users and 8 stations (Study A, head-to-head instance). The stations have a baseline capacity of 2000 L ±40%. Unary filtering and AC-3 are pre-applied to prune initial domains.

What we observe. Due to aggressive rationing policy constraints (such as plate-parity and capacity reserves), only 8 of the 12 users are servable, leaving 4 users unassignable due to domain wipe-outs. Both solvers successfully find a feasible assignment for the servable subset. Backtracking search finds the solution in 0.14 ms, expanding 9 nodes with 0 backtracks, and returns a load-balanced assignment at a cost of 30,295. Min-Conflicts local search solves the same instance in 0.03 ms, requiring only a single repair iteration to converge on a slightly lower cost of 29,253 due to direct optimization of the soft objective equation (11). The results are summarized in Table 11; the resulting station loads are verified in Figure 6, and this assignment is mapped onto real Dhaka roads in Figure 8 (left).

Solver	Found?	Cost	Served / Unassign.	Effort	Runtime (ms)
Backtracking (MRV+LCV+FC)	Yes	30,295	8 / 4	9 nodes	0.14
Min-Conflicts	Yes	29,253	8 / 4	1 iter	0.03

Table 11: Solver performance on the Dhanmondi worked test case (12 users, 8 stations; Study A).

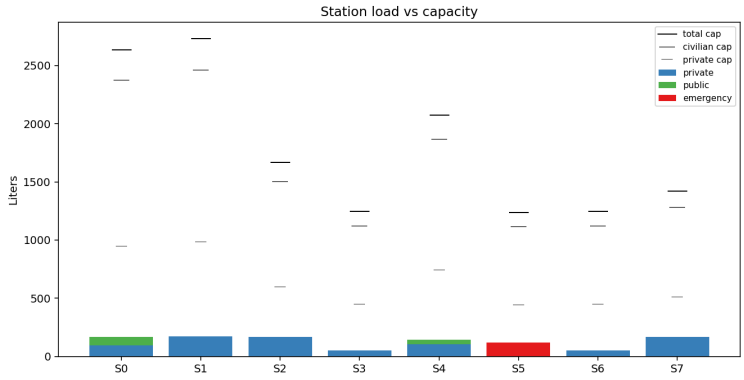


Figure 6: Per-station demand vs. capacity under the backtracking assignment (worked example). The private/public/emergency stacks stay within each station's civilian cap and strategic reserve, i.e. constraints G3/G4/G8 hold.

A. Heuristic Benefit and Solver Comparison. The three-way comparison (Table 12) evaluates naive backtracking (heuristics off), heuristic backtracking, and Min-Conflicts on identical instances of increasing size. For small instances ($n \leq 40$), all three methods solve instantly with zero backtracks. At $n = 70$, naive backtracking collapses under the combinatorial weight, timing out after 601,618 backtracks. In contrast, heuristic backtracking solves the same instance in 4.12 ms with *zero* backtracks, demonstrating the efficiency of MRV and forward checking. At $n = 100$, even heuristic backtracking times out as systematic search cannot escape the exponential scaling, while Min-Conflicts returns a high-quality approximate assignment in 957 ms. This highlights the classic trade-off: systematic search guarantees completeness but is exponential, while local search scales to larger regimes [27].

n	Naive backtracking	Heuristic backtracking	Min-Conflicts
8	solved, 0 bt, 0.08 ms	solved, 0 bt, 0.12 ms	solved, 2 it, 0.10 ms
20	solved, 0 bt	solved, 0 bt	solved, 1 it
40	solved, 0 bt	solved, 0 bt	solved, 1 it
70	**timeout**, 601,618 bt	solved, 0 bt, 4.12 ms	solved, 4 it, 0.99 ms
100	**timeout**	**timeout**, 150,549 bt	approx., 3,136 it, 957 ms

Table 12: Assignment 2 three-way comparison (Study A). "bt" = backtracks, "it" = Min-Conflicts iterations.

Heuristic Component Analysis. To isolate each heuristic's contribution, we run an ablation study by adding them cumulatively on a hard, *over-constrained (infeasible)* instance (Table 13). MRV and the degree heuristic prune about 6% of backtracks by identifying domain failures early. However, adding LCV and forward checking does not reduce backtracks further on this infeasible instance, instead increasing runtime due to per-node overhead. This contrast is instructive: on a feasible instance, heuristics guide the search directly to a solution (Table 12), but on an infeasible instance, the search space must be fully exhausted. Ordering heuristics (LCV) and lookahead (FC) accelerate finding solutions, but do not speed up proving their absence.

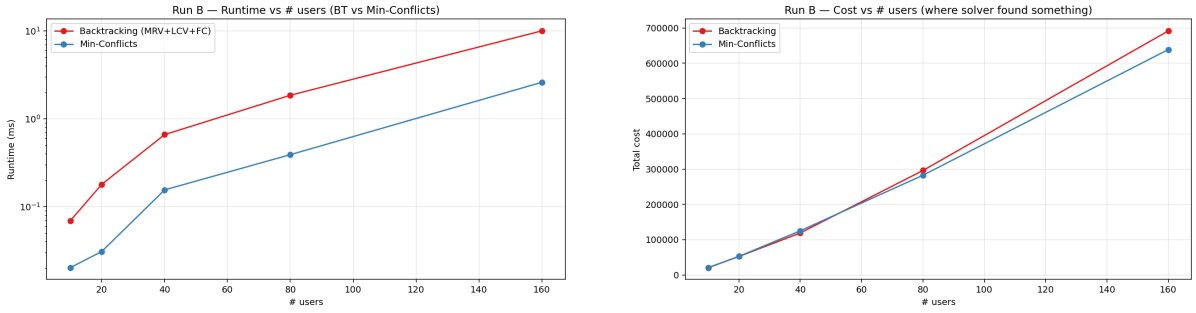
Configuration	Backtracks	Runtime (ms)
naiv backtracking	121,545	9,001
+ MRV + degree	114,504	9,901
+ LCV	114,504	12,365
+ forward checking	114,504	13,025

Table 13: Heuristic ablation on a hard (infeasible) instance, heuristics added cumulatively.

B. Systematic vs. Local Search at Scale. The scaling sweep (Table 14, Figure 7) measures performance for $n \in \{10, 20, 40, 80, 160\}$ on a larger multi-area bounding box. Where both solvers succeed, Min-Conflicts is consistently faster (e.g., 2.60 ms vs. 9.99 ms at $n = 160$) and returns solutions of comparable or slightly lower cost. While both runtimes scale smoothly in this range, the three-way comparison shows that systematic search hits a wall between $n = 70$ and $n = 100$, whereas local search scales reliably.

n	BT time (ms)	MC time (ms)	BT cost	MC cost
10	0.07	0.02	20,882	20,941
20	0.18	0.03	52,439	52,722
40	0.66	0.16	119,093	125,316
80	1.85	0.39	296,237	282,730
160	9.99	2.60	691,841	638,971

Table 14: Assignment 2 scaling sweep (Study B): runtime and cost of backtracking (BT) vs. Min-Conflicts (MC).

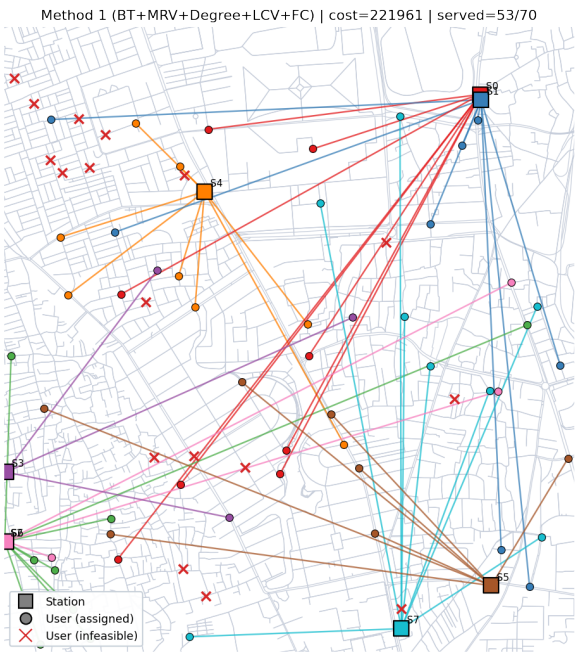


(a) Runtime vs. number of users n.

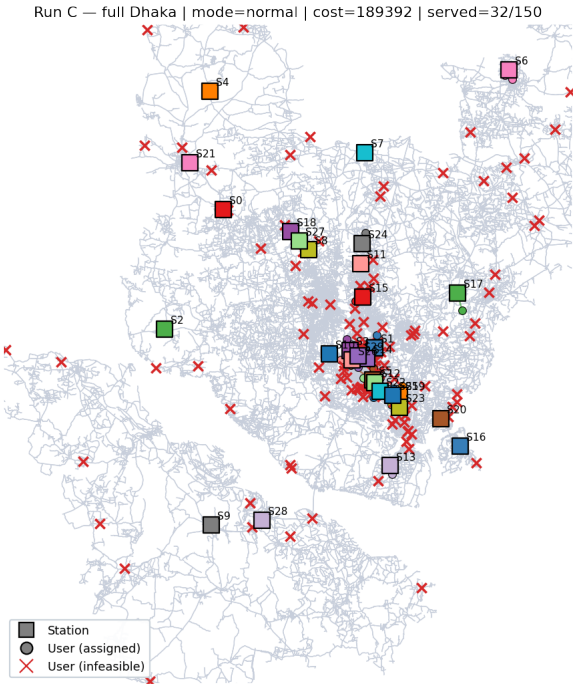
(b) Solution cost vs. n.

Figure 7: Scaling sweep: backtracking vs. Min-Conflicts as the number of users grows. Min-Conflicts is faster throughout while matching solution cost.

Study C Policy Validation under Aggressive Rationing. The aggressive rationing policy restricts feasibility rather than solver behavior. On a realistic city-scale instance of 150 users and 30 stations, only 32 users are servable, leaving 118 unassignable. This is not a solver failure, but a policy constraint: plate-parity wipe-outs, emergency reserves, CNG closures, and capacity limits empty user domains before search begins. Figure 8 (right) visualizes this sparse city-wide assignment. The CSP framework thus acts as a policy validator, proving that severe rationing cannot achieve full service coverage.



(a) Worked example: 12 users, 8 stations (Dhanmondi).



(b) Full-Dhaka assignment (150 users, 30 stations).

Figure 8: Color-coded user-to-station assignments on real Dhaka roads at two scales: a small worked example (left) and the full city (right), where aggressive rationing leaves 118/150 users unassignable a feasibility limit of the policy, not a solver failure.

4.4. Key Takeaways and Empirical Synthesis

Our evaluation of the CSOP framework for urban fuel rationing yields three primary insights:

1. **Heuristic Pruning Eliminates Combinatorial Explosion**: AC-3 arc consistency, MRV variable ordering, and forward checking turn intractable depth-first backtracking into an instant solver for feasible instances ($n \leq 70$, 0 backtracks). On infeasible instances, however, ordering heuristics add per-node overhead without reducing backtracks, revealing that lookahead accelerates *finding* solutions rather than *proving* inconsistency.
2. **Systematic vs. Local Search Scalability**: Systematic backtracking hits an absolute combinatorial wall between $n = 70$ and $n = 100$ ($> 150{,}000$ backtracks, timing out). In contrast, Min-Conflicts local search scales smoothly up to $n = 160$ (2.60 ms runtime), directly optimizing the soft load-balancing cost equation (11) while sacrificing completeness guarantees.
3. **Rationing Feasibility Bounds**: On a full city-scale instance (150 users, 30 stations), strict plate-parity, emergency reserves, and queue caps render 118 users unassignable due to domain wipe-outs before search even commences. The CSP framework thus acts as a formal policy validator, proving that severe supply rationing cannot achieve full service coverage.

While this CSOP addresses static, discrete multi-user assignment under hard policy constraints, urban services often require continuous spatial placement under non-submodular objectives or dynamic adaptive policies. In Section 5, we transition to population-based optimization (CCTV camera placement).

5. Population-Based Optimization for Surveillance-Camera Placement

5.1. Problem Formulation

Public-safety surveillance is a resource-allocation problem under a hard budget: a city can afford only a fixed number of cameras, yet must watch the roads and public spaces where incidents are most likely and most costly. Dhaka makes this concrete. The Dhaka Metropolitan Police is rolling out a city-wide municipal CCTV network and integrating a large number of private cameras into a central monitoring grid, and densely monitored zones such as Gulshan–Baridhara are reported to experience lower crime than sparsely covered areas [42, 46]. Unlike blanket coverage, a real deployment must decide not only *where* to mount each unit but *which way it faces*, since a camera sees only a directional cone. The planning question we study is therefore: *given a limited camera budget, where and facing which direction should each camera be placed to watch the most important and highest-risk roads?* We formalize this as a directional Maximal Covering Location Problem (MCLP), which is NP-hard [5].

Inputs. A Dhaka neighbourhood road graph; a demand field of weighted points p (road samples weighted by road importance, junctions by degree, boosted near real OSM points-of-interest, and modulated by a crime/crowd risk field); a fixed camera budget K; and a fleet of directional cameras, each with a field-of-view angle FOV and range R.

Output. A placement that assigns to each camera k an intersection n_k and a facing angle θ_k (the decision variables). A demand point p is *covered* iff it lies within some camera's range *and* field-of-view cone.

Objective. Among placements, maximize weighted coverage, optionally penalizing redundant overlap:

$$\mathcal{F}(\{n_k, \theta_k\}) \;=\; \underbrace{\textstyle\sum_{p:\, c_p \geq 1} w_p}_{\text{weighted coverage}} \;-\; \lambda \underbrace{\textstyle\sum_p w_p \, \max(0,\, c_p - 1)}_{\text{overlap penalty}}, \tag{12}$$

where c_p is the number of cameras that see point p, w_p its weight, and λ the overlap penalty. With $\lambda = 0$ the objective is *submodular* (plain coverage); with $\lambda > 0$ redundant coverage is penalized and the objective becomes *non-submodular* a distinction that turns out to decide which algorithm wins. Table 15 collects the notation.

Symbol	Meaning
K	camera budget (number of cameras to place)
R, FOV	camera range and field-of-view angle
(n_k, θ_k)	intersection and facing angle of camera k (the decision variables)
w_p	weight of demand point p = road importance × POI × crime factor
c_p	number of cameras covering p
λ	overlap penalty (0 = submodular; > 0 = non-submodular)

Table 15: Assignment 3 notation. Numeric settings for every symbol are justified in Section 5.2.2.

We treat this study as a controlled empirical comparison of three population-based optimizers against a greedy baseline, analysed in Section 5.3, targeting three questions. First, on a *submodular* coverage objective (λ=0), whether the population-based methods (GA, PSO, ACO) improve on a greedy baseline. Second, when an overlap penalty makes the objective *non-submodular* (λ>0), whether a metaheuristic overtakes greedy, and which mechanism is responsible. Third, whether the synthetic crime/crowd layer steers placement toward high-risk zones, and how coverage transfers across Dhaka neighbourhoods.

5.2. Methodology

5.2.1 Simulation Environment and Demand Model

A neighbourhood sub-graph is cropped from the Dhaka OSM graph obtained through OSMnx [4]. Road segments are sampled into *demand points*, each weighted by three multiplicative factors: the OSM importance of the road it lies on (arterials outrank service lanes), the degree of the nearest intersection (busy junctions outrank dead-ends), and a proximity boost for points near real OSM points-of-interest (schools, markets, hospitals, bus stands). A synthetic *crime/crowd risk field* a sum of Gaussian "risk" clusters, part of them seeded on real POIs and the rest placed pseudo-randomly, fully determined by the run seed further multiplies the weights so that placement is pulled toward high-risk zones. Camera coverage is modelled as a directional cone: a point p is covered by camera k iff the Euclidean line-of-sight distance $\|p - n_k\| \leq R$ *and* the bearing of p from n_k lies within $\theta_k \pm \text{FOV}/2$. Line-of-sight (rather than road-network) distance is used here because a camera sees across open space, unlike a car in Assignments 1–2 which must follow the road graph.

5.2.2 Design Rationale and Empirical Basis

A coverage model is only as credible as the reasoning behind its terms, so we justify each numeric choice rather than merely assert it. Every value below is a design parameter of the simulated environment, not a fitted measurement.

(i) Demand weighting. The multiplicative weight $w_p = (\text{road importance}) \times (\text{junction degree}) \times (\text{POI proximity}) \times (\text{crime factor})$ encodes the real surveillance priority behind the DMP rollout: busy arterials, crowded junctions, and high-footfall public sites matter more than quiet residential lanes [42, 46]. Only the *ordinal* structure is asserted an arterial junction near a market outweighs a back-street not the precise multipliers.

(ii) Crime/crowd risk field. Because open, edge-level crime data for Dhaka is not publicly available at the required granularity, we *synthesize* a risk field rather than fit one. It is a sum of Gaussian clusters (baseline: a handful of clusters, roughly half anchored on real POIs and half placed pseudo-randomly, each with a few-hundred-metre spread), deterministic per seed so every solver sees an identical instance. This mirrors the earlier assignments' stance: the field is a transparent stand-in whose *shape* elevated risk around crowded hubs is what matters, not its absolute magnitudes.

(iii) Camera physics. The fleet mixes two sensor types whose reach reflects commodity CCTV optics: wide-angle 90° units with a 200 m effective range, and narrower 120° units with a 130 m range (a wider cone trades range for breadth). These are the physical constants of the coverage cone; the range-sensitivity sweep of Section 5.3 scales them explicitly to show the model responds as expected.

(iv) Budget and fleet. The baseline budget is K=15 cameras for a typical neighbourhood, raised to K=30 for the dense adversarial regime. The fleet is a fixed mix of the two sensor types. Both are operational choices a planner sets from a procurement budget the model treats K as a free knob and reports how coverage scales with it.

(v) Overlap penalty λ. The penalty λ in equation (12) is the single lever that converts a pure (submodular) coverage objective into a redundancy-averse (non-submodular) one. We treat it as the primary experimental control and sweep it over a range from 0 (plain coverage) up to values large enough that avoiding double-coverage dominates, precisely to locate where a metaheuristic starts to matter.

(vi) Optimizer representation and controls. The three population-based optimizers differ along three deliberate axes how they represent the facing angle θ, their search operators, and the controls they share summarized in Table 16. Two controls are common to all three: a *memetic warm start* [31] (one individual / particle / ant path is seeded with the greedy baseline layout,[1] so no metaheuristic can score below greedy), and *early stopping* with a patience of 20 under a cap of 200 iterations (500 in the adversarial study). The single decisive representation choice is the facing angle: only PSO carries θ as a *continuous* variable, whereas ACO samples 8 fixed angle bins and GA mutates θ in discrete jumps the crux of why PSO alone can trim the fine overlap that a penalty punishes.

Optimizer	Angle θ	Representation	Key hyper-parameters
GA	discrete jumps	K genes (*node*, θ)	pop 40, tournament 3, crossover 0.85 (uniform), mutation 0.2 (move node / rotate $\theta \sim \mathcal{N}(0, 0.6)$), elitism 2
PSO	**continuous**	continuous $(x, y, \theta)^K$, snapped to nearest node	swarm 30, inertia w=0.7, cognitive c_1=1.5, social c_2=1.5
ACO	8 fixed bins	subset of K nodes + angle bin	ants 20, α=1 (pheromone), β=2 (heuristic η), evaporation ρ=0.1, elitist deposit

Table 16: Per-optimizer representation and hyper-parameters (identical across all runs; tunable design settings, not fitted values). All three are warm-started from the greedy baseline.

On the simulation framing. This environment is explicitly a *simulation*: none of the demand weights, crime clusters, camera ranges, or solver hyper-parameters is a measured Dhaka quantity, and each can be replaced by a calibrated real-data estimate without touching the optimization or evaluation code. The framework is fully modular, and every empirical claim we make depends only on *relative* solver behaviour which method wins, and under what objective structure not on the absolute value of any single constant.

5.2.3 From Coverage Model to Optimization: The Algorithmic Approach

Before detailing each optimizer, it is worth being precise about *how* the coverage objective connects to the search machinery because one representation choice in particular is exactly what separates the three methods.

What every optimizer receives. All three optimizers operate on the same instance and call the same objective $\mathcal{F}(\{n_k, \theta_k\})$ from equation (12) to score any complete K-camera layout. None of them has gradient access; $\mathcal{F}$ is a black box evaluated by counting which weighted demand points fall inside which cones. Each is *warm-started* from the greedy baseline layout (Section 5.2.2), so each begins from an already-strong solution it can only improve on. What they do *not* share is how they represent the facing angle θ (Table 16), and as the three descriptions below make clear that single difference is the whole story.

Genetic Algorithm. The GA [16] evolves a population of K-gene chromosomes (Algorithm 5). Tournament selection, uniform crossover, and mutation (move a camera to another node, or rotate its angle) explore combinations the baseline never considers but because its angle mutations are *discrete jumps*, it cannot finely tune a facing direction.

[1] The greedy baseline is not one of the population methods under study; it builds a layout one camera at a time, each time adding the (node, one-of-16-directions) camera of largest marginal gain in $\mathcal{F}$, using 16 fixed directions and one camera per node. It is fast and, on a submodular objective, near-optimal by the classic $(1 - 1/e)$ guarantee, so we use it as a strong starting point and comparison baseline.

Algorithm 5 Genetic Algorithm (chromosome = K genes, each a $(node, \theta)$ pair).

population $P \leftarrow$ random K-camera layouts, one seeded with the greedy layout
for generation $= 1, 2, \ldots$ (until stagnation) **do**
    evaluate fitness $\mathcal{F}$ equation (12) of every layout; keep the top e (elitism)
    while P not refilled **do**
        $p_1, p_2 \leftarrow$ tournament-select; child $\leftarrow$ uniform crossover of p_1, p_2
        mutate: move a camera to a random node *or* rotate its θ; repair one-per-node
    end while
end for
return best layout

Particle Swarm Optimization. PSO [22] treats a layout as a point in continuous $(x, y, \theta)^K$ space (Algorithm 6). Each particle is pulled toward its own and the swarm's best positions; the (x, y) coordinates snap to the nearest intersection, but the angle θ stays *continuous*. This is the one optimizer that can dial a facing direction to any value the property that lets it trim overlap the discrete methods cannot.

Algorithm 6 Particle Swarm Optimization (particle = continuous $(x, y, \theta)^K$, snapped to nearest node).

initialize positions x_i, velocities v_i; seed one particle with greedy; set $pbest_i, gbest$
for iteration $= 1, 2, \ldots$ (until stagnation) **do**
    for all particle i **do**
        $v_i \leftarrow w\, v_i + c_1 r_1 (pbest_i - x_i) + c_2 r_2 (gbest - x_i)$; $x_i \leftarrow \text{clip}(x_i + v_i)$
        decode: snap each (x, y) to its nearest intersection; evaluate $\mathcal{F}$
        update $pbest_i$ and $gbest$
    end for
end for
return $gbest$

Ant Colony Optimization. ACO [10] builds layouts by sampling K distinct nodes with probability $\propto \tau^\alpha \eta^\beta$ (pheromone $\times$ single-camera desirability), then an angle bin per node (Algorithm 7). Pheromone reinforces good regions across iterations, but its angles are drawn from 8 fixed bins again discrete.

Algorithm 7 Ant Colony Optimization (subset selection of K intersections + discrete angles).

pheromones $\tau_{\text{node}}, \tau_{\text{ang}} \leftarrow 1$; heuristic $\eta_c =$ best single-camera coverage of node c; seed best from greedy
for iteration $= 1, 2, \ldots$ (until stagnation) **do**
    for all ant **do**
        build a layout: pick K distinct nodes with prob. $\propto \tau_{\text{node}}^\alpha \eta^\beta$, and an angle bin per node
        evaluate $\mathcal{F}$
    end for
    evaporate $\tau \leftarrow (1 - \rho)\tau$; deposit $\propto \mathcal{F}$ on the best-so-far layout (elitist)
end for
return best layout

The greedy baseline. For reference, the three optimizers are all measured against a simple greedy baseline (used above only as their warm-start seed, not as a method under study). It

builds a layout one camera at a time, each step adding the (node, one-of-16-directions) camera of largest marginal gain in $\mathcal{F}$ and never revisiting an earlier choice. On a submodular objective this myopia is nearly harmless the classic $(1 - 1/e)$ guarantee makes it near-optimal but once it clusters cameras at the busiest hub it cannot undo the resulting redundancy, which is precisely the weakness the penalized regime exposes.

When $\lambda = 0$ the objective is submodular and the greedy baseline is already near-optimal, so warm-started metaheuristics can only tie it adding search machinery buys nothing. The moment $\lambda > 0$ makes the objective non-submodular, redundant overlap must be actively trimmed, and trimming it well requires *fine* control of the facing angle. Only PSO's continuous θ provides that, which predicts that PSO and *only* PSO should overtake greedy in the penalized regime. This is the central empirical question of the assignment, and Section 5.3 tests it directly.

5.2.4 Evaluation Metrics and Experimental Protocol

Our study demonstrates,

- **Weighted coverage (%)** the fraction of total demand weight seen by at least one camera; the primary quality measure.
- **Overlap (%)** the weight counted redundantly ($c_p \geq 2$); the quantity the penalty λ targets.
- **Objective** the penalized score $\mathcal{F}$ of equation (12) that the solvers actually maximize.
- **Runtime** wall-clock seconds to converge.
- **Iterations** generations / swarm steps / ant rounds until early stopping.

Controlled sweeps vary one factor at a time while holding the graph, fleet, and solver settings fixed: the crime layer on/off, the camera budget K, the camera range, the neighbourhood, and critically the overlap penalty λ. To expose greedy's myopia decisively, we add a deliberately adversarial "greedy-fail" scenario: a tiny 500 m area packed with 30 very-long-range cameras, where a single camera already covers a large share of demand, so avoiding redundant overlap is the entire game. The overall workflow is summarized in Figure 9.

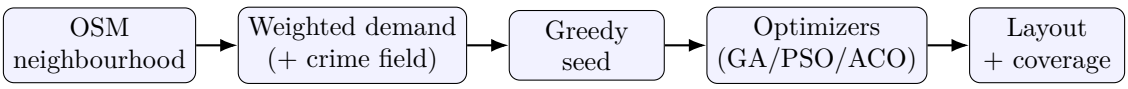


Figure 9: Workflow: a weighted, crime-aware demand field defines the coverage objective; a greedy layout seeds the metaheuristics, which refine node placement and facing angles.

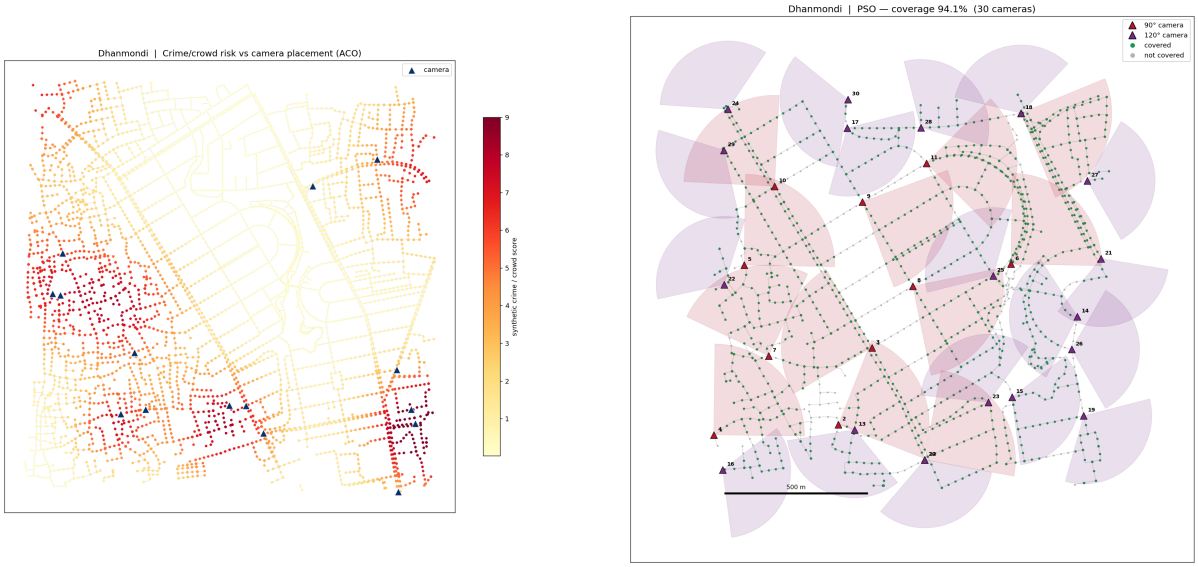
5.3. Results and Discussion

Worked test case

Scenario. Before the aggregate sweeps, we walk through one concrete instance: place K=15 cameras in Dhanmondi with the crime/crowd layer enabled and no overlap penalty (λ=0). The greedy baseline first builds a layout by repeatedly adding the camera of largest marginal coverage gain; each optimizer is warm-started from that layout and is then free to move nodes and rotate facing angles.

The chosen cameras concentrate on the high-risk red zones of the crime field (Figure 10, left) rather than spreading uniformly, and the PSO solution's field-of-view wedges tile the busy arterials and junctions of the real Dhanmondi road graph (right). Turning the crime layer on lifts weighted coverage from 21.9% to 25.3% as cameras migrate toward risk direct evidence that the weighted-demand model steers placement where a real deployment would want it. On this submodular instance all three optimizers converge to exactly the same 25.3% as the greedy seed:

none can improve on an already near-optimal baseline. Whether that tie is fundamental or an artefact of this easy regime is what the aggregate sweeps below resolve.



(a) Crime/crowd risk field with chosen cameras.

(b) PSO placement with field-of-view wedges.

Figure 10: Worked test case (Dhanmondi, K=15, crime layer on): cameras concentrate in high-risk red zones (left); the PSO solution and its coverage cones on real Dhaka roads (right).

Aggregate Analysis

We now generalize from this single instance to controlled multi-scenario sweeps covering three regimes: the submodular baseline, the non-submodular penalty regime, and crime steering with neighbourhood transfer.

For plain coverage ($\lambda = 0$, $K = 15$) all three optimizers tie the greedy baseline at $\approx 25.3\%$ weighted coverage (Table 17). This is expected: coverage is submodular, and greedy enjoys the classic $(1 - 1/e) \approx 63\%$ optimality guarantee, so the warm-started metaheuristics can only match it. A metaheuristic is *not* automatically better an honest and important baseline.

Metric	GA	PSO	ACO
Weighted coverage (%)	25.3	25.3	25.3

Table 17: Baseline (Dhanmondi, K=15, λ=0): all three optimizers tie the greedy baseline (25.3%) on submodular coverage.

PSO overtakes greedy. Once an overlap penalty makes the objective non-submodular (dense scenario, $K = 30$, long range), PSO *consistently* exceeds greedy at every penalty level (Table 18), while GA and ACO whose angle moves are discrete only match it. The mechanism is specific: PSO optimizes the *continuous* facing angle θ, which greedy and the discrete methods can only sample in fixed steps, so PSO trims the redundant overlap they cannot. To expose this decisively we designed an adversarial "greedy-fail" scenario a tiny 500 m area packed with 30 very-long-range cameras, where a single camera already covers $\sim$40% (Table 19). Here greedy's myopia is fatal: it commits cameras to the highest-demand hub and cannot undo the resulting redundancy, so its *overlap stays stuck at* $\sim$4.5%; PSO spreads the cameras out, cutting overlap below 1.5% and beating greedy on coverage at every penalty. This is the clearest evidence that a metaheuristic genuinely outperforms greedy once the objective punishes redundancy the myopic

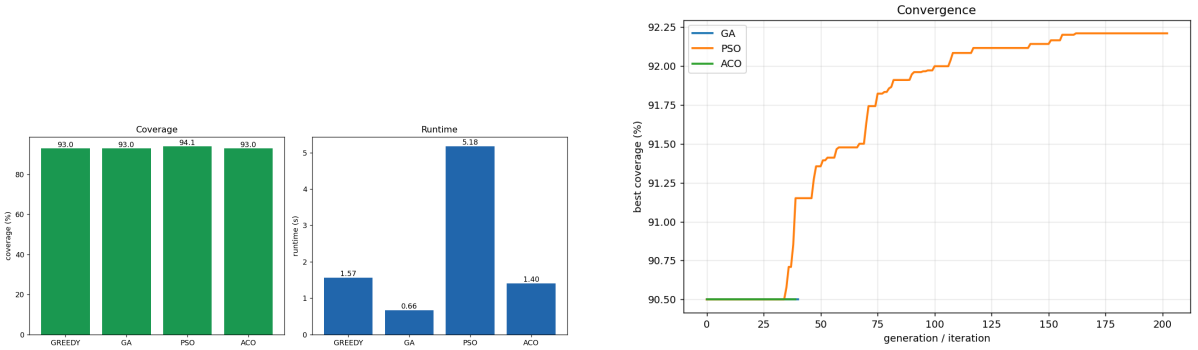
baseline cannot escape mirroring Assignment 2's ablation lesson that the advanced method wins only when the problem *structure* defeats the baseline's guarantee. Figure 11 shows the solver comparison and convergence.

λ	Greedy	GA	PSO	ACO
0.0	98.01	98.01	**98.56**	98.01
0.5	96.40	96.40	**97.24**	96.40
1.0	94.71	94.71	**95.55**	94.71
2.0	93.04	93.04	**94.06**	93.04

Table 18: Dense, non-submodular regime (K=30): weighted coverage (%). PSO beats greedy at every overlap penalty λ; GA and ACO (discrete angles) only match greedy.

	Greedy		PSO (winner)	
λ	coverage %	overlap %	coverage %	overlap %
5	90.6	4.5	**93.6**	1.5
10	90.6	4.5	**91.9**	0.8
20	92.8	4.4	**93.0**	1.0

Table 19: Adversarial "greedy-fail" scenario (500 m area, 30 long-range cameras): coverage *and overlap*. Greedy stays clustered at the busiest hub with ∼4.5% wasted overlap; PSO spreads cameras out, cutting overlap below 1.5% and winning coverage at every penalty.



(a) Coverage & runtime by solver.

(b) Convergence (coverage vs iteration).

Figure 11: Solver comparison (left) and convergence curves (right) in the dense non-submodular regime.

Crime steering and neighbourhood transfer. The crime-steering effect quantified in the worked test case weighted coverage rising from 21.9% to 25.3% once the risk layer is enabled (Figure 10) holds across instances. Coverage also transfers sensibly across neighbourhoods (Table 20): Gulshan is easiest to cover (36.3%) and Mirpur hardest (24.1%). Reassuringly, this matches the real-world report that Gulshan's dense camera coverage coincides with lower crime [46] the model's easy-to-cover neighbourhood is the one that is, in practice, well covered.

Neighbourhood	Dhanmondi	Gulshan	Old Dhaka	Uttara	Mirpur
Coverage (%)	25.3	**36.3**	31.4	30.1	24.1

Table 20: Neighbourhood transfer (weighted coverage %, K=15).

Sensitivity to physical parameters. Coverage is governed far more by the cameras' physical reach than by the choice of optimizer. Scaling the 90° range from 140 to 260 m lifts weighted coverage from 18.2% to 32.9%, and because plain coverage is submodular all three optimizers return *identical* values at each range (Table 21). Search budget shows the expected diminishing returns: PSO edges from 25.25% at 20 iterations to 25.48% at 200, while GA and ACO stay flat. The practical implication is that, in the submodular regime, longer-range (or more) cameras pay off more than a fancier optimizer the metaheuristic only earns its keep once the objective is non-submodular.

90° camera range (m)	140	200	260
Weighted coverage (%)	18.2	25.3	32.9

Table 21: Sensitivity: weighted coverage (%) vs. camera range (all three optimizers return identical values under submodular coverage).

5.4. Key Takeaways and Empirical Synthesis

This study shows precisely when population-based optimization is worth its cost. On submodular coverage, greedy already achieves near-optimal placement and the warm-started metaheuristics can only tie it an honest baseline that resists the temptation to declare a metaheuristic automatically superior. The swarm advantage emerges only when an overlap penalty makes the objective non-submodular *and* a continuous decision variable (the facing angle) is available: there PSO's continuous search overtakes greedy while the discrete GA and ACO do not, and the adversarial greedy-fail scenario isolates the mechanism greedy's myopic overlap stays stuck while PSO spreads the cameras out. Finally, the weighted-demand model, including the synthetic crime field, steers placement toward the high-risk zones that real Dhaka deployments target, and coverage transfers sensibly across neighbourhoods. The overarching lesson mirrors Assignment 2's ablation finding: the advanced method earns its keep only when the problem's *structure* defeats the baseline's guarantee.

6. Reinforcement Learning for Stochastic Multi-Stop Delivery

6.1. Motivation and Problem Formulation

On-demand delivery is now woven into daily life in Dhaka food, parcels, and tiffin services dispatch riders across the city continuously yet the same two forces that dominate the earlier assignments make each trip uncertain: traffic congestion that collapses average travel speed to a crawl at peak [44, 50], and monsoon waterlogging that can render residential streets impassable within minutes [47]. A rider dispatched with several drop-offs one of them urgent must decide, at every junction and under this uncertainty, which way to head so as to complete all deliveries at the least expected cost while serving the urgent order first. Unlike the deterministic routing of Assignment 1, the outcome of each action here is *stochastic*, so the agent needs not a single path but a *policy*: a mapping from situation to action. We formalize this as a Markov Decision Process (MDP) (S, A, T, R, γ) [3, 41].

Inputs. A Dhaka neighbourhood road graph with per-edge travel times; a set of K delivery stops (some flagged urgent); and hazard labels marking a fraction of roads as jammed or flooded. The MDP components are:

- **State** $s = (v, U)$: current node v and the set $U \subseteq \mathcal{T}$ of not-yet-visited stops (an immutable `frozenset` for hashing). There are $|V| \cdot 2^K$ states, so K is kept small to avoid state-space explosion.
- **Action** a: a neighbouring node to head toward.
- **Transition** $T(s, a, s')$: *stochastic*, depending on road type a normal road usually reaches

the intended node, while a jammed or flooded road may divert or stall the agent (values in Section 6.2.2).

- **Reward** R: a per-move travel-time cost, a hazard penalty for a divert/stall, a delivery bonus for reaching a stop (larger if urgent), and a completion bonus when all stops are served (magnitudes in Section 6.2.2).
- **Discount** γ: makes earlier rewards worth more, so urgency is encoded via reward magnitude and discounting rather than a state-expanding deadline.

Output. An optimal policy $\pi^* : S \to A$ that maximizes the expected discounted return:

$$\pi^* \;=\; \arg\max_{\pi} \; \mathbb{E}_{\pi}\left[\sum_{t=0}^{\infty} \gamma^t \, r_t\right], \tag{13}$$

where r_t is the reward at step t and the expectation is taken over the stochastic transitions T under policy π. Executing π^* yields a delivery route that serves the urgent stop first. Table 22 lists the notation.

Symbol	Meaning
$s = (v, U)$	state: current node v, unvisited stops U
$K = \lvert\mathcal{T}\rvert$	number of delivery stops; state count $= \lvert V\rvert \cdot 2^K$
$T(s, a, s')$, R, γ	transition, reward, discount factor
$u(s)$, $Q(s, a)$	state value (VI) and action value (Q-Learning)
π^*	optimal policy (state $\mapsto$ action)

Table 22: Assignment 4 notation. Numeric settings are justified in Section 6.2.2.

We treat this study as a controlled empirical comparison of model-based planning against model-free learning on one stochastic routing problem, analysed in Section 6.3, targeting three questions.

First, whether *model-free* Q-Learning recovers the *model-based* Value-Iteration optimum, and at what computational cost.

Second, how sensitive the policy is to the discount γ, and where Q-Learning becomes unstable.

Third, whether reward magnitude plus discounting correctly encodes delivery *urgency*, and how the number of stops and hazards affect the solution and its tractability.

6.2. Methodology

6.2.1 Simulation Environment and Hazard Model

A sub-graph is cropped from the Dhaka OSM graph [4]. Each directed edge is assigned a deterministic travel time from its physical length and a road-type speed. Two hazard types then make travel *stochastic*: a fraction of primary/secondary roads are marked *jam*, and a fraction of residential/tertiary roads are marked *flood*, both seeded so every agent faces an identical instance. On a normal road the agent almost always reaches the node it heads for; on a jammed road it may be diverted to an unintended neighbour; on a flooded road it may stall in place and lose the move. These outcomes define the transition $T(s, a, s')$, and each unwanted outcome carries a penalty in R.

6.2.2 Design Rationale and Empirical Basis

The MDP is only as credible as the numbers behind its transitions and rewards, so we justify each rather than assert it. Every value is a design parameter of the simulated environment.

(i) Stochastic transitions. Each road type carries its own reach probability, mirroring the two forces that actually make Dhaka travel uncertain. A *normal* road almost always delivers the agent to its intended node (0.95, with a 0.05 chance of slipping to a random neighbour). *Traffic*: average speed collapses to ∼7 km/h in peak congestion, so a jammed arterial (primary/secondary road) frequently diverts a driver off the intended turn [44, 50] we model this as a 0.70 reach / 0.30 divert split. *Waterlogging*: monsoon rain routinely submerges residential and low-lying streets until they are impassable, stranding vehicles [47] we model a flooded road (residential/tertiary) as a 0.50 reach / 0.50 stall split. Assigning jams to arterials and floods to smaller, low-lying roads matches the reported pattern.

(ii) Reward magnitudes. The reward structure asserts only an *ordinal* priority: completing all deliveries dominates (+1000), reaching an individual stop is worth a solid bonus (+200, doubled to +400 when urgent), a divert or stall is mildly punished (−50), and every move costs its travel time. These magnitudes are chosen so that the terminal goal is worth pursuing through several hazardous moves, not calibrated to any currency.

(iii) Urgency without a deadline. Encoding urgency as a *larger reward* rather than a hard time deadline is a deliberate modelling choice: a deadline would add a time dimension to the state and multiply the state count, whereas a reward bonus keeps the space at $|V| \cdot 2^K$ while still driving the policy to serve urgent stops first.

(iv) Discount and agent controls. Both agents share the discount $\gamma = 0.95$ (its effect is swept in Section 6.3). Value Iteration stops when the Bellman residual falls below $\epsilon(1-\gamma)/\gamma$; Q-Learning adds a learning rate α, an ϵ-greedy decay schedule, and a convergence test (mean reward stable over three windows of 100 episodes) so it can early-stop rather than always exhausting its episode budget. Table 23 collects the settings.

Agent	Knowledge	Hyper-parameters
Value Iteration	model-based (knows T, R)	γ=0.95; stop threshold ϵ=1.0 ($\delta < \epsilon(1-\gamma)/\gamma$); $\leq$ 1000 sweeps
Q-Learning	model-free (samples T, R)	α=0.1, γ=0.95, $\epsilon : 1.0 \rightarrow 0.05$ (exp. decay), $\leq$ 3000 episodes $\times$ 200 steps; convergence over 3×100-episode windows

Table 23: Per-agent design considerations and hyper-parameters (tunable settings, not fitted values).

On the simulation framing. This environment is a *simulation*: the transition probabilities, reward magnitudes, hazard densities, and learning hyper-parameters are transparent design choices, none of them a measured Dhaka quantity, and each can be replaced by a calibrated estimate without touching the agents. Our empirical claims depend only on relative behaviour whether model-free learning recovers the model-based optimum, and where it becomes fragile not on the absolute value of any constant.

6.2.3 From MDP to Policy: The Algorithmic Approach

Before detailing the two agents, it is worth being precise about the single factor that separates them: whether the agent is handed the environment's model (T, R) or must discover it from experience.

What both agents receive. Both agents act in the same MDP, over the same state space S and action set A, discount future reward by the same γ, and are judged by the same greedy-policy return. Neither is told the optimal policy; both must compute it.

The model is the decisive axis. What they do *not* share is access to the model (T, R), contrasted in Table 23: Value Iteration is handed it and plans exactly, whereas Q-Learning must discover it through sampled experience. As the two descriptions below make clear, this single difference decides both the cost of solving and the robustness of the result.

Value Iteration (model-based). Knowing T and R, Value Iteration can compute the exact expected value of every action by summing over all successor states, and iterates the Bellman optimality update [3] (Equation (14)) to convergence before extracting the greedy policy (Algorithm 8). It never needs to explore it plans.

$$u^{t+1}(s) \;\leftarrow\; r(s) \;+\; \gamma \max_{a} \sum_{s'} T(s,a,s')\, u^{t}(s'), \tag{14}$$

Algorithm 8 Value Iteration.

$u(s) \leftarrow 0 \;\; \forall s$; threshold $\vartheta \leftarrow \epsilon(1-\gamma)/\gamma$
repeat
  $\delta \leftarrow 0$
  for all non-terminal s **do**
    $u' \leftarrow r(s) + \gamma \max_a \sum_{s'} T(s,a,s')\, u(s')$
    $\delta \leftarrow \max(\delta, |u' - u(s)|)$; $u(s) \leftarrow u'$
  end for
until $\delta < \vartheta$
$\pi^*(s) \leftarrow \arg\max_a \sum_{s'} T(s,a,s')\, u(s') \;\; \forall s$; **return** π^*

Q-Learning (model-free). Knowing neither T nor R, Q-Learning learns action values from sampled experience, following an ϵ-greedy policy with decaying exploration and bootstrapping each estimate from the next state's current best value (Equation (15), Algorithm 9). It must balance exploring unknown actions against exploiting what it has already learned.

$$Q(s,a) \;\leftarrow\; Q(s,a) + \alpha\big[\, r + \gamma \max_{a'} Q(s',a') - Q(s,a) \,\big]. \tag{15}$$

Algorithm 9 Q-Learning (ϵ-greedy, decaying ϵ, with a convergence check).

$Q(s,a) \leftarrow 0$; $\epsilon \leftarrow \epsilon_0$
for episode $= 1$ to $E_{\max}$ **do**
  $s \leftarrow$ start;
  for step $= 1$ to L **do**
    $a \leftarrow$ random w.p. ϵ, else $\arg\max_{a'} Q(s,a')$ ▷ ϵ-greedy
    $(s', r) \leftarrow \textsc{Step}(s,a)$ ▷ sample from T, R
    $Q(s,a) \mathrel{+}= \alpha\,[\, r + \gamma \max_{a'} Q(s',a') - Q(s,a) \,]$; $s \leftarrow s'$
    if s terminal **break**
  end for
  $\epsilon \leftarrow \max(\epsilon_{\min}, \epsilon \cdot d)$ ▷ decay
  every W episodes: stop if mean reward stable for 3 windows
end for
$\pi(s) \leftarrow \arg\max_a Q(s,a)$; **return** π

Why model availability decides everything. When the model is known, Value Iteration is both exact and cheap a few dozen sweeps suffice. When it is not, Q-Learning can still recover the same optimal policy, but only by paying for experience (many thousands of episodes) and by being more fragile to hyper-parameters: with a moderate discount the bootstrapped targets can fail to propagate a sparse terminal reward. This predicts the two central findings model-free *recovers* model-based at a compute premium, and destabilizes at a low γ where the exact planner does not. Section 6.3 tests both.

6.2.4 Evaluation Metrics and Experimental Protocol

Our study shows,:

- **Route reward** the discounted return of the learned greedy policy; the primary quality measure.
- **Iterations / episodes** Bellman sweeps (VI) or training episodes (QL) to converge.
- **Runtime** wall-clock seconds to convergence.
- **Delivery order** the sequence in which stops are served, inspected to verify urgent-first behaviour.

Controlled sweeps vary one factor at a time on identical instances (Dhanmondi, 88 nodes): the discount γ, the number of stops K, the urgent-stop count, the hazard density, the episode budget, and the random seed. Value Iteration and Q-Learning are run on every configuration so their policies and costs can be compared directly. The overall workflow is summarized in Figure 12.

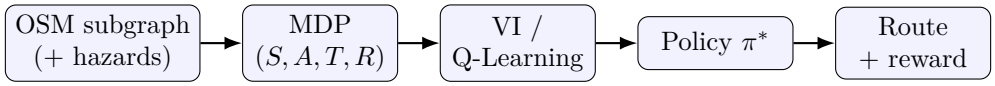


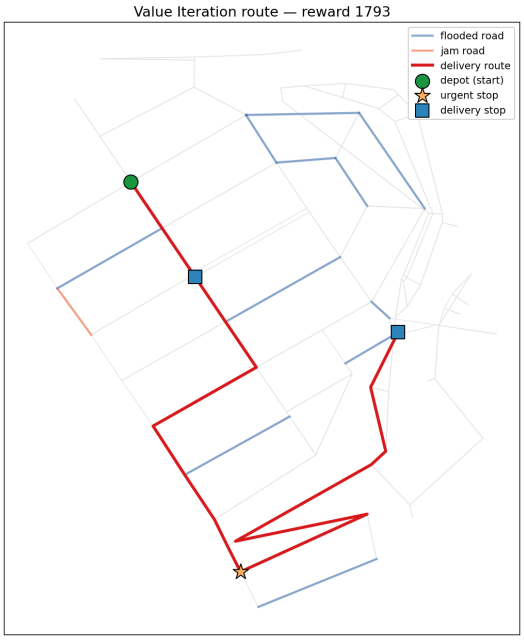
Figure 12: Workflow: a hazard-aware MDP is solved by model-based Value Iteration or model-free Q-Learning to yield a delivery policy and its executed route.

6.3. Results and Discussion

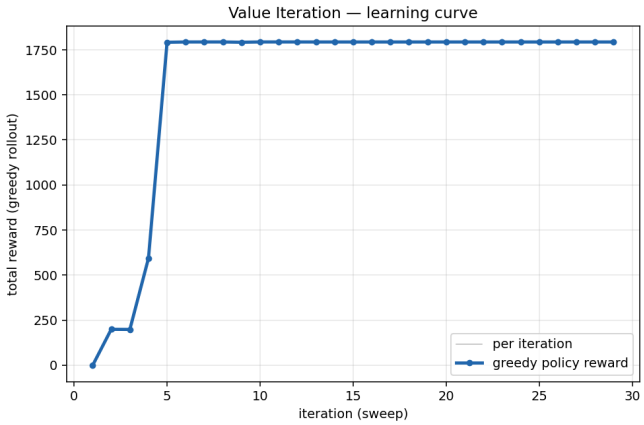
6.3.1 Worked test case: one delivery run

Scenario. Before the aggregate sweeps, we walk through a single instance: a Dhanmondi sub-network (88 nodes) with 3 delivery stops, one of them urgent, medium hazard density, and the default discount γ=0.95. Both agents solve the same instance Value Iteration by planning over the known model, Q-Learning by sampling episodes with no model at all.

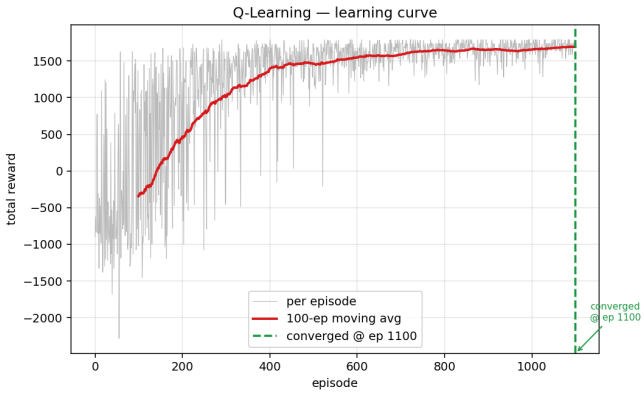
What we observe. Both agents produce essentially the same policy: the agent routes through jammed (orange) and flooded (blue) roads to reach the *urgent* stop first, then completes the remaining two (Figure 13, left), earning a route reward of 1787 (VI) versus 1785 (QL). The two paths are visually indistinguishable model-free learning has recovered the model-based optimum. What differs sharply is the cost of getting there (Figure 13, right): Value Iteration converges in 33 Bellman sweeps (0.15 s), while Q-Learning needs ∼3000 episodes ($\approx$ 3.2 s) to climb to the same reward. This single run previews the assignment's central contrast same answer, ∼20× the compute which the sweeps below confirm and stress-test.



(a) Optimal delivery route (urgent stop first).



(b) Value Iteration convergence (reward per sweep).



(c) Q-Learning learning curve (reward per episode).

Figure 13: Worked test case (Dhanmondi, 3 stops, 1 urgent): the policy routes through jammed (orange) and flooded (blue) roads to deliver the urgent stop first (left); Value Iteration converges in a few dozen sweeps while Q-Learning climbs to the same reward over thousands of episodes (right).

Aggregate Analysis

We now generalize from this single run to controlled sweeps that vary one factor at a time, examining three aspects: model-free versus model-based cost, sensitivity to the discount γ, and urgency, stops, and tractability.

Model-free recovers model-based, at a compute premium. The tie previewed above holds across every configuration (Table 25): Value Iteration and Q-Learning converge to essentially the same route reward at each setting, and both always deliver the urgent stop first confirming that a model-free learner, given enough experience, recovers the model-based optimum and that reward-plus-discounting correctly encodes urgency without a deadline. The invariant difference is cost: the $\sim$20$\times$ compute premium of Q-Learning over Value Iteration is the classic model-based/model-free trade-off [41].

The discount γ is the critical knob. γ governs foresight, and it is the most sensitive hyper-parameter (Table 24). At $\gamma = 0.5$ the distant +1000 terminal reward is discounted into near-irrelevance, so *both* agents turn myopic (reward collapses to 399). More strikingly, at $\gamma = 0.8$ Value Iteration is fine (1788) but Q-Learning *destabilizes* to 399: with a moderate discount and a fixed learning rate, the bootstrapped Q targets fail to propagate the sparse terminal reward

reliably within the episode budget a concrete, honest illustration that model-free learning is more fragile to hyper-parameters than exact planning. From $\gamma \geq 0.9$ both agree again.

γ	VI reward	QL reward
0.50	399	593
0.80	1788	**399**
0.90	1787	1785
0.95	1787	1785
0.99	1787	1787

Table 24: Discount γ sweep. QL destabilizes at γ=0.8.

Setting	VI	QL
baseline (3 stops)	1787	1785
2 stops (352 states)	1596	1596
4 stops (1408 states)	1987	1987
0 urgent	1587	1587
2 urgent	1987	1987

Table 25: VI vs. QL agree across stops and urgency (reward; iters/episodes).

Urgency, stops, and tractability. Reward scales interpretably (Table 25): each additional stop adds ∼200 (1596 → 1787 → 1987 for 2 → 3 → 4 stops), and converting a normal stop to urgent adds another ∼200 (1587 → 1787 → 1987 for 0 → 1 → 2 urgent), exactly as the reward table dictates, with the urgent stop always served first. The results are robust across seeds (1787–1794) and hazard densities (1792/1787/1788 for low/medium/high). The cost of the extra stop is state-space growth: the state count $|V| \cdot 2^K$ rises from 352 to 1408 as K goes 2 → 4, which is why K must stay small for exact Value Iteration the practical ceiling of a tabular method.

6.4. Key Takeaways and Empirical Synthesis

This assignment contrasts planning and learning on one stochastic problem. Given the model, Value Iteration plans the optimal delivery policy in a few dozen sweeps; without the model, Q-Learning learns the *same* policy including urgent-first delivery from experience alone, but at ∼20× the compute and with greater hyper-parameter fragility, exposed by its γ=0.8 collapse where the exact planner is untroubled. Reward magnitude plus discounting encodes urgency correctly without a state-expanding deadline, and reward scales interpretably with stops and urgency. The discount γ is the decisive knob, and the tabular state count $|V| \cdot 2^K$ is the scalability ceiling the point at which one would move from exact Value Iteration to function-approximation reinforcement learning.

7. Conclusions

Holding the domain fixed the Dhaka road network and varying the Artificial Intelligence paradigm produced four findings that reinforce one another.

In **search** (Section 3), A^* matched the uniform-cost optimum while expanding 4.7× fewer nodes (7,519 versus 35,496), but a controlled two-setup comparison showed that this advantage is not a property of A^* at all: replacing the reverse-Dijkstra lower bound with a geometry-only Euclidean heuristic blind to traffic and safety, the terms that dominate the cost surface collapsed A^* onto uniform-cost search (35,116 expansions) and inflated greedy best-first's solution cost more than fourfold. Informed search is worth exactly as much as its heuristic's fidelity to the true cost, and no more.

In **constraint satisfaction** (Section 4), AC-3 propagation with MRV, degree, and LCV ordering plus forward checking converted a 601,618-backtrack timeout into a zero-backtrack solve at $n = 70$, yet systematic search still hit an absolute wall by $n = 100$, where only Min-Conflicts returned a usable assignment. An ablation on a deliberately infeasible instance sharpened the lesson: ordering and lookahead accelerate *finding* a solution but do nothing to accelerate *proving* that none exists. At city scale the framework acted less as a solver than as a policy validator, showing that strict plate-parity, emergency reserves, and queue caps leave 118 of 150 users unassignable before search even begins a limit of the rationing policy, not of the algorithm.

In **population-based optimisation** (Section 5), the honest result is that on plain submodular coverage a greedy baseline is already near-optimal by the classic $1 - 1/e$ guarantee, and all three warm-started metaheuristics could only tie it. The swarm advantage appeared only once an overlap penalty made the objective non-submodular *and* a continuous decision variable was available: Particle Swarm Optimisation, alone in optimising the facing angle continuously, overtook greedy at every penalty level, while the discrete-angle Genetic Algorithm and Ant Colony Optimisation did not. The adversarial greedy-fail scenario isolated the mechanism precisely greedy's wasted overlap stayed pinned near 4.5% while PSO drove it below 1.5%.

In **reinforcement learning** (Section 6), model-based Value Iteration and model-free Q-Learning converged to the same optimal policy, both serving the urgent delivery first, confirming that reward magnitude combined with discounting encodes urgency without a state-expanding deadline. The invariant difference was cost: Q-Learning paid a $\sim$20$\times$ compute premium for not being handed the model, and proved measurably more fragile, destabilising at $\gamma = 0.8$ where the exact planner was untroubled.

The unifying lesson. Algorithm selection should follow problem structure rather than habit or novelty: *informativeness* for heuristic search, *propagation and scale* for constraint reasoning, *submodularity and parameter continuity* for optimisation, and *model availability under uncertainty* for sequential decision making. In three of the four cases the sophisticated method earned its keep only when a specific structural property defeated the simpler baseline's guarantee; recording the cases where it did *not* is as much a result as recording the cases where it did.

Limitations. Three deserve emphasis. First, several components of the environment are synthesised because open Dhaka data at edge-level granularity does not exist station capacities, the crime and crowd risk field, hazard placement, and the traffic and safety proxies of the routing study so the absolute cost and coverage magnitudes given here are not measurements. Second, the Markov decision process is kept deliberately small ($|V| \cdot 2^K$ states) to keep tabular Value Iteration exact, which caps the number of delivery stops well below an operational fleet's. Third, the coverage model treats visibility as a clean geometric cone with no occlusion, which overstates achievable coverage in the dense, irregular built form of Old Dhaka in particular.

Future work. The natural extensions follow directly from those limitations: substituting real incident and crash records for the synthetic risk field and calibrating the cost model against measured travel times; adding line-of-sight occlusion, and ideally building-footprint geometry, to the coverage objective; and replacing tabular Value Iteration with function-approximation reinforcement learning to lift the state-space ceiling on the delivery task and admit a realistic number of stops. A further direction is to relax the assumption that the four problems are independent: in a real deployment the routing, rationing, surveillance, and delivery decisions share the same congested network and could be posed jointly.

References

[1] Salim Abdesselam and Zine-Eddine Baarir. Three variants particle swarm optimization technique for optimal cameras network two dimensions placement. *Journal of Applied Engineering Science and Technology*, 4(1):9–16, 2018. doi: 10.69717/jaest.v4.i1.56.

[2] Roman Barták. Constraint programming: In pursuit of the holy grail. *Proceedings of the Week of Doctoral Students (WDS'99), MatfyzPress, Prague*, pages 555–564, 1999.

[3] Richard Bellman. *Dynamic Programming*. Princeton University Press, 1957.

[4] Geoff Boeing. OSMnx: New methods for acquiring, constructing, analyzing, and visualizing

complex street networks. *Computers, Environment and Urban Systems*, 65:126–139, 2017. doi: 10.1016/j.compenvurbsys.2017.05.004.

[5] Richard Church and Charles ReVelle. The maximal covering location problem. *Papers of the Regional Science Association*, 32(1):101–118, 1974. doi: 10.1111/j.1435-5597.1974.tb00902.x.

[6] Maurice Clerc and James Kennedy. The particle swarm—explosion, stability, and convergence in a multidimensional complex space. *IEEE Transactions on Evolutionary Computation*, 6(1):58–73, 2002. doi: 10.1109/4235.985692.

[7] Rina Dechter. *Constraint Processing*. Morgan Kaufmann, 2003.

[8] Edsger W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1:269–271, 1959. doi: 10.1007/BF01386390.

[9] Marco Dorigo and Luca Maria Gambardella. Ant colony system: A cooperative learning approach to the traveling salesman problem. *IEEE Transactions on Evolutionary Computation*, 1(1):53–66, 1997. doi: 10.1109/4235.585892.

[10] Marco Dorigo, Vittorio Maniezzo, and Alberto Colorni. Ant system: Optimization by a colony of cooperating agents. *IEEE Transactions on Systems, Man, and Cybernetics, Part B*, 26(1):29–41, 1996. doi: 10.1109/3477.484436.

[11] F. Fausto, G. Corona, A. Gonzalez, and M. Pérez-Cisneros. Metaheuristics-assisted placement of omnidirectional image sensors for visually obstructed environments. *Biomimetics*, 10(9):579, 2025. doi: 10.3390/biomimetics10090579.

[12] Zong Woo Geem, Joong Hoon Kim, and G. V. Loganathan. A new heuristic optimization algorithm: Harmony search. *Simulation*, 76(2):60–68, 2001. doi: 10.1177/003754970107600201.

[13] David E. Goldberg. *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley, 1989. doi: 10.5555/534133.

[14] Robert M. Haralick and Gordon L. Elliott. Increasing tree search efficiency for constraint satisfaction problems. *Artificial Intelligence*, 14(3):263–313, 1980. doi: 10.1016/0004-3702(80)90051-X.

[15] Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968. doi: 10.1109/TSSC.1968.300136.

[16] John H. Holland. *Adaptation in Natural and Artificial Systems*. University of Michigan Press (2nd ed., MIT Press, 1992), 1975. https://direct.mit.edu/books/monograph/2574/.

[17] Ekram Hossain, Md. Rezaul Karim, Mir Hasan, Syeed Abrar Zaoad, Tauhid Tanjim, and Md. Mosaddek Khan. SPaFE: A crowdsourcing and multimodal recommender system to ensure travel safety in a city. *IEEE Access*, 10:71221–71232, 2022. doi: 10.1109/ACCESS.2022.3187964.

[18] Ronald A. Howard. *Dynamic Programming and Markov Processes*. MIT Press, 1960.

[19] Zangir Iklassov, Ikboljon Sobirov, Ruben Solozabal, and Martin Takáč. Reinforcement learning for solving stochastic vehicle routing problem. In *Proceedings of the 15th Asian Conference on Machine Learning*, volume 222, pages 502–517. PMLR, 2024.

[20] Dervis Karaboga. An idea based on honey bee swarm for numerical optimization. Technical Report TR06, Erciyes University, Engineering Faculty, Computer Engineering Department, 2005. https://abc.erciyes.edu.tr/pub/tr06_2005.pdf.

[21] Sourabh Katoch, Sumit Singh Chauhan, and Vijay Kumar. A review on genetic algorithm: Past, present, and future. *Multimedia Tools and Applications*, 80(5):8091–8126, 2021. doi: 10.1007/s11042-020-10139-6.

[22] James Kennedy and Russell Eberhart. Particle swarm optimization. In *Proceedings of the IEEE International Conference on Neural Networks*, volume 4, pages 1942–1948, 1995. doi: 10.1109/ICNN.1995.488968.

[23] Scott Kirkpatrick, C. Daniel Gelatt, and Mario P. Vecchi. Optimization by simulated annealing. *Science*, 220(4598):671–680, 1983. doi: 10.1126/science.220.4598.671.

[24] Richard E. Korf. Depth-first iterative-deepening: An optimal admissible tree search. *Artificial Intelligence*, 27(1):97–109, 1985. doi: 10.1016/0004-3702(85)90084-0.

[25] Vipin Kumar. Algorithms for constraint-satisfaction problems: A survey. *AI Magazine*, 13 (1):32–44, 1992. doi: 10.1609/aimag.v13i1.976.

[26] Alan K. Mackworth. Consistency in networks of relations. *Artificial Intelligence*, 8(1): 99–118, 1977. doi: 10.1016/0004-3702(77)90007-8.

[27] Steven Minton, Mark D. Johnston, Andrew B. Philips, and Philip Laird. Minimizing conflicts: A heuristic repair method for constraint satisfaction and scheduling problems. *Artificial Intelligence*, 58(1–3):161–205, 1992. doi: 10.1016/0004-3702(92)90007-K.

[28] Seyedali Mirjalili and Andrew Lewis. The whale optimization algorithm. *Advances in Engineering Software*, 95:51–67, 2016. doi: 10.1016/j.advengsoft.2016.01.008.

[29] Seyedali Mirjalili, Seyed Mohammad Mirjalili, and Andrew Lewis. Grey wolf optimizer. *Advances in Engineering Software*, 69:46–61, 2014. doi: 10.1016/j.advengsoft.2013.12.007.

[30] George L. Nemhauser, Laurence A. Wolsey, and Marshall L. Fisher. An analysis of approximations for maximizing submodular set functions—i. *Mathematical Programming*, 14(1):265–294, 1978. doi: 10.1007/BF01588971.

[31] Ferrante Neri and Carlos Cotta. Memetic algorithms and memetic computing optimization: A literature review. *Swarm and Evolutionary Computation*, 2:1–14, 2012. doi: 10.1016/j.swevo.2011.11.003.

[32] Judea Pearl. *Heuristics: Intelligent Search Strategies for Computer Problem Solving*. Addison-Wesley, 1984.

[33] Ira Pohl. Heuristic search viewed as path finding in a graph. *Artificial Intelligence*, 1(3–4): 193–204, 1970. doi: 10.1016/0004-3702(70)90007-X.

[34] Martin L. Puterman. *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. John Wiley & Sons, 1994. doi: 10.1002/9780470316887.

[35] R. Venkata Rao, V. J. Savsani, and D. P. Vakharia. Teaching-learning-based optimization: A novel method for constrained mechanical design optimization problems. *Computer-Aided Design*, 43(3):303–315, 2011. doi: 10.1016/j.cad.2010.12.004.

[36] Esmat Rashedi, Hossein Nezamabadi-Pour, and Saeid Saryazdi. GSA: A gravitational search algorithm. *Information Sciences*, 179(13):2232–2248, 2009. doi: 10.1016/j.ins.2009.02.016.

[37] Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, 4 edition, 2021. http://aima.cs.berkeley.edu/.

[38] Oren Salzman, Ariel Felner, Carlos Hernández, Han Zhang, Shao-Hung Chan, and Sven Koenig. Heuristic-search approaches for the multi-objective shortest-path problem: Progress

and research opportunities. In *Proceedings of the 32nd International Joint Conference on Artificial Intelligence (IJCAI)*, pages 6759–6768, 2023. doi: 10.24963/ijcai.2023/757.

[39] Yuhui Shi and Russell Eberhart. A modified particle swarm optimizer. In *Proceedings of the IEEE International Conference on Evolutionary Computation*, pages 69–73, 1998. doi: 10.1109/ICEC.1998.699146.

[40] Rainer Storn and Kenneth Price. Differential evolution – a simple and efficient heuristic for global optimization over continuous spaces. *Journal of Global Optimization*, 11(4):341–359, 1997. doi: 10.1023/A:1008202821328.

[41] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2 edition, 2018. http://incompleteideas.net/book/the-book-2nd.html.

[42] The Business Standard. DMP to install 11,000 CCTV cameras in Dhaka, 700 in Mohammadpur. https://www.tbsnews.net/bangladesh/dmp-install-11000-cctv-cameras-dhaka-700-mohammadpur-1419846, 2023. Accessed: July 2026.

[43] The Business Standard. Road accidents claim 7,902 lives in 2023. https://www.tbsnews.net/bangladesh/transport/road-accidents-claim-7902-lives-2023-774622, 2024. Bangladesh Jatri Kalyan Samity data; Accessed: July 2026.

[44] The Daily Star. Traffic jam in Dhaka eats up 3.2m working hours everyday: World Bank. https://www.thedailystar.net/city/dhaka-traffic-jam-congestion-eats-32-million-working-hours-everyday-world-bank-1435630, 2018. Accessed: July 2026.

[45] The Daily Star. Fuel pumps run dry across districts. https://www.thedailystar.net/environment/natural-resources/energy, 2022. On fuel rationing during the supply crisis. Accessed: July 2026.

[46] The Daily Star. DMP to bring capital under CCTV coverage. https://www.thedailystar.net/news/bangladesh/news/dmp-bring-capital-under-cctv-coverage-3304471, 2023. Accessed: July 2026.

[47] The Daily Star. 76mm of rain leaves parts of Dhaka waterlogged, disrupts daily life. https://www.thedailystar.net/news/bangladesh/news/76mm-rain-leaves-parts-dhaka-waterlogged-disrupts-daily-life-4221826, 2025. Accessed: July 2026.

[48] The Wire. Bangladesh's fuel queues vanished overnight after price hike, exposing a deeper energy policy failure. https://thewire.in/economy/bangladeshs-fuel-queues-vanished-overnight-after-price-hike, 2022. Accessed: July 2026.

[49] Christopher J. C. H. Watkins and Peter Dayan. Q-learning. *Machine Learning*, 8(3–4): 279–292, 1992. doi: 10.1007/BF00992698.

[50] World Bank. Towards great Dhaka: A new urban development paradigm eastward. Technical report, World Bank, Washington, DC, 2018. https://openknowledge.worldbank.org/handle/10986/29925.

[51] Xin-She Yang. Firefly algorithms for multimodal optimization. *Stochastic Algorithms: Foundations and Applications*, 5796:169–178, 2009. doi: 10.1007/978-3-642-04944-6_14.

[52] Xin-She Yang and Suash Deb. Cuckoo search via lévy flights. In *World Congress on Nature & Biologically Inspired Computing*, pages 210–214, 2009. doi: 10.1109/NABIC.2009.5393690.